

CO3201 Computer Science Project

Animating Facial Expressions with AI Mood Detection

Dissertation

SCHOOL OF COMPUTING AND MATHEMATICAL
SCIENCES, UNIVERSITY OF LEICESTER

Richard Yaya-Abatan

Submitted: May 2025

Table of Contents

CO3201 COMPUTER SCIENCE PROJECT	1
1. DECLARATION	3
2. ABSTRACT	4
3. INTRODUCTION	5
3.1. Aims	5
3.2. Objectives	5
3.3. Keywords	6
3.4. Requirements	7
4. RELATED WORK	8
5. TECHNICAL SPECIFICATION	10
5.1. HTML5, JavaScript and CSS	10
5.2. Flask	10
5.3. Py-Feat	10
5.4. Roundcubemail	11
5.5. Delanuary Triangulation	11
5.6. OpenCV	11
5.7. Gmail IMAP/SMTP	11
5.8. PIL	11
5.9. Imageio	11
6. ARCHITECTURE AND DESIGN	13
6.1. UML Core Components	14
6.2. Architecture Assumptions and workflow Summary	16
6.3. Core Components – linked to UML	16
6.4. Rationale and Design Choices	17
6.5. Limitations of Design	17
7. METHODOLOGY	18
7.1. Survey Design	18
7.2. System Design	23
8. IMPLEMENTATION	26
8.1. System Development	26
8.2. Optimisation and Performance	31
8.3. Challenges	32
9. TESTING	36
9.1. Testing Strategies:	36
9.2. Limitations of Testing	38
10. SYSTEM EVALUATION AND FUTURE IMPROVEMENTS	39
10.1. Evaluate Against the Requirements	39
10.2. Limitations and Future Improvements	40
11. CONCLUSION	42
12. BIBLIOGRAPHY AND CITATIONS	43
13. APPENDIX	47

1. Declaration

All sentences or passages quoted in this report, or computer code of any form whatsoever used and/or submitted at any stages, which are taken from other people's work, have been specifically acknowledged by clear citation of the source, specifying author, work, date and page(s).

Any part of my own written work, or software coding, which is substantially based upon other people's work, is duly accompanied by clear citation of the source, specifying author, work, date and page(s).

I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this module and the degree examination as a whole.

Name: Richard Yaya-Abatan

Date: 02/05/2025

Signed:

A handwritten signature in black ink, appearing to read 'Rich', written over the word 'Signed:'.

2. Abstract

Efficient communication is a fundamental foundation of any business relationship; the impact of poor communication can be detrimental to core business principles, e.g., productivity [27]. Current tools to aid in the provision of this are lacklustre or fail to make way for this to happen at all. The use of non-textual means of communication in professional workspaces, e.g., Graphics Interchange Format (GIF) and pictograms like emojis to bolster emotional context and tone, is very easily misconstrued or misinterpreted. Even whilst studies have shown their lack of use may very easily lead to business inefficiencies and, in rare cases, escalation of conflict [27]. I discovered that, with research supporting their usage in building relationships that extend outside of the work environment, the perception of the user relies heavily on the receiver and the type of content used. I surveyed 30 employees of different companies to gain an insight into their understanding of the impact these non-textual additives may have on our ability to forge relationships, both in the workplace and outside environments. Reviewing the data collected proved a conflict between the purpose and appropriateness of available resources, necessitating the creation of a revolutionary application that could break present stigmas. This was completed by utilising the agile model to develop a plugin for creating topic-specific GIFs using live-captured images, which participants viewed as a possible tool for offering clarity and a mood-driven tone in writing emails.

3. Introduction

A common saying is, "Conflict is as natural to human life as breathing" [1]. When bunched together, it is almost impossible for conflict not to materialise in a group or organisation, especially exaggerated in high-stakes, fast-paced work environments. This project addresses a key origin of conflicts: a lack of emotional and physical context leading to a breakdown in workplace communications. A survey of 1400 staff members conducted by Fierce Inc. shows that 86 percent of respondents blame inefficient communication as the source of workplace failure [2]. In a world where most internal and external business communication still happens over email, with 57 percent of businesses using email as their number one client communication tool, 36 percent using it for internal communications and 42 percent of people still working from home post-COVID-19 [3]. It is more important now than ever to create resources that allow efficient and accurate online.

3.1. Aims

My software mainly aims to create a method of inputting live-generated mood-influenced GIFs into email composition. The use of GIFs to add tone to emails or other forms of communication is not revolutionary, as major platforms provide support for ".gif" files in the format of attachments or inline injections. However, current means of GIF generation require an online GIF generator or simply using the limited built-in GIF picker provided by the user's device operating system, if such native emoticon/GIF support is available at all. I aim to develop a system that takes two live capture images as inputs, processes these images and returns the detected mood and a bespoke animated GIF. A method of customisation that ensures accurate tone portrayal in written communication. The final aim is to integrate my application into Roundcube web mail, an open-source email service that will allow users to attach the output of my system into real emails [8].

3.2. Objectives

- Configure an IMAP platform in which users can access and manage their personal emails.
- Build a Flask server to receive and process API calls from a web client.
- Implement browser-based webcam capture.
- Create a method of detecting facial expression from a live capture image.
- Develop a means of morphing images into animations.
- Create a way to present backend responses to the client e.g., detected mood, animated GIF.
- Attach the generated animation to the email composition.
- Process regeneration – granting the user repeated usage of the application with differing outcomes.

As a result of time constraints and a lack of necessary resources, the process of morphing a single image into an animated GIF as initially proposed has been dropped in favour of morphing two live-captured images into an animation.

3.3. Keywords

- IMAP: Internet Message Access Protocol – used to access and manage emails stored on a mail server from any location or device without the need for storage or downloads [4].
- SMTP: Simple Mail Transfer Protocol – a communication protocol that uses Transmission Control Protocol to securely send email messages [5]
- Flask: a web framework used for building web applications and presenting the modal data to the user in a specific format [6][7].
- Roundcube Webmail: an open-source email service that enables users to access their email account and allows creation of custom plugins [8].
- Py-feat: a toolbox that detects facial expression data from images and videos, allowing for analysis and presentation [9].
- Image Morphing: the process of seamlessly transforming between images by creating multiple sub-frames that blend to create an animation [10].
- Delaunay Triangulation: used to create corresponding points between two images by aligning their triangular mesh [11].
- Action Unit (AU): A unit key that identifies the facial movement of a person, represented by a float number that can be mapped to a dictionary of emotions.

3.4. Requirements

3.4.1. Non-Functional

3.4.1.1. Performance:

- Detection Latency: The process of mood detection via Py-feat must be completed in under 5 seconds.
- GIF Generation: The Facial animation pipeline must finish in under 5 seconds.

3.4.1.2. Usability:

- Progress Feedback: The front-end user interface provides clear status updates or errors.
- Cancellation: The user should be able to interrupt/abort the process.
- The system must allow animation regeneration by re-capturing two new expressions without restarting the session.

3.4.1.3. Accuracy:

- Mood Accuracy: The mood detection model should be able to identify the dominant emotion with a consistency of 70 percent.

3.4.1.4. Compatibility:

- Operating system (OS) support: The Roundcube plugin and Flask backend should run on multiple OS.
- Browser Support: Frontend Client should be compatible with most modern browsers, e.g., Chrome and Safari.

3.4.1.5. Security & Privacy:

- Temporary File Cleanup: All temporary images and GIFs must be deleted after cleanup or on exit.
- Origin Validation: postMessage handlers must verify that data is sent to and from the client's device.

3.4.1.6. Maintainability:

- Code structure should be modular.

3.4.2. Functional

- Capture live images through either a webcam or a built-in camera.
- Ability to attach the animated GIF to the draft email.
- A range of emotions/moods should be detectable via Py-feat.
- Animation of captured images into a GIF.
- Live access to email servers.
- Grant GIF export abilities to both the local device and other sources.
- Error detection should be handled gracefully with clear error messages.

4. Related Work

The most obvious existing solution to my system is the use of emojis. Emojis provide the same physical cues as GIFs, where suitable, they can add a world of context to communications, as well as providing intent. Even in the presence of negative feedback, studies have shown that the inclusion of an emoji can soften the perception of communication [12]. However, public opinion of emojis, particularly in the workplace, remains a topic of speculation, especially when used by higher-ups. In 2020, researchers Riordan and Glikson conducted two large-scale experiments with U.S. office workers to dive into the topic. In the first study, it was discovered that male participants scored their managers' likeability and effectiveness higher when they used emojis. In contrast, female participants had no strong feelings about the use of emojis on their manager's likeability, but a negative impact on effectiveness. The second study revealed that female participants viewed emails with emojis in a positive light, but also as less appropriate, especially in a formal context [13]. These findings reflect the vagueness around the use of emojis; their impact is largely dependent on conditions being perfect, such as the recipient's gender and the environment of communication, even then, it seems almost impossible for it to be perceived as entirely positive.

The alternative to this is simple GIFs, which predate emojis by 12 years since their creation in 1987 by Steve Wilhite [14]. A recent experiment conducted by K. Sukeerthi compared a group of Gen-Z students to students who graduated in the 1990s, Group A's use of GIFs in communication reflected a childish appearance on the user, as it may be seen as informal because it reflects a lack of "deep knowledge" as it "lowers the dignity in the line spoken". However, it allowed for more frequent, expressive and less verbose communication compared to Group B's more formal approach, where any informal text led to misunderstandings [15]. Coupled with the fact that in most instances where GIFs are used, users are forced to pick from a limited selection, the likelihood of the included GIF being deemed inappropriate for the context is much higher.

In the face of criticisms, the usage of emoticons and GIFs remains essential to establishing interpersonal bonds in a professional context. As Miltner and Highfield demonstrate, the animated GIF allows users to illustrate shared cultural knowledge in a single, concise visual snippet, an ability that emoticons fail to mirror [16]. Similarly, the COVID-19 pandemic led to an accelerated rise of remote workspaces, and as many companies chose to adopt this format of working post-pandemic, it meant that for many new staff, their introduction to their colleagues happened remotely. This meant that establishing interpersonal relationships required more than just regular conversation, as shown by Esha Shandilya, who found that newcomers to virtual teams who chose to deliberately and progressively adopt non-textual responses were able to convey emotion accurately and align with emergent

group norms whilst building rapport with unfamiliar colleagues, even with the connotation of an informal and more personal setting [17].

Combined, these studies suggest that despite concerns about professionalism or appropriateness, non-textual forms of communication such as these remain practitioners' tool of choice for "ice-breaking". These limitations are areas my mood-based GIF application seeks to overcome by offering users the option of contextually grounded animations rather than relying on readily available one-size-fits-all emoticons and GIFs.

5. Technical Specification

5.1. HTML5, JavaScript and CSS

Industry-standard web technologies for client-side functionality in browsers. Their purpose is to provide interactive browser-based user interfaces and access webcam hardware from within the browsers

5.2. Flask

A lightweight, well-documented web framework that is ideal for building small-scale web services that host complex applications [18]. It runs a local server that exposes HTTPS endpoints (e.g., '/camera') used for the image capture workflow and image/animation processing. It handles POST requests from the frontend, housing and performing the image analysis and animation generation processes. It was chosen for its minimalistic setup and flexibility in combination with detailed documentation and community support, making it ideal for getting started quickly. Alternatives like Django were considered unnecessarily complex for the REST-style endpoints required.

5.3. Py-Feat

A free, open-source Python package for automated expression recognition and facial landmarks in images and video format. This Python library implements facial expression analysis of static images using pre-trained machine or deep learning models like RetinaFace for face detection, ResMaskNet for emotion detection and MobileFaceNet for facial landmark detection. The package is trained on a large mixed dataset of labelled images ranging from 4k -1.1m images per dataset.

Facial analysis happens in five stages:

- Face Detection:
- Landmark detection
- Face and Head position estimation
- Action unit detection
- Emotion Detection

The 'detector' class groups together multiple computer vision models. This was chosen because of its simplicity and freedom to use compared to other models, such as OpenFace, its landmark detection ability is later used in the animation process. Its purpose in my application is to process images from the frontend by identifying facial landmarks in both neutral and emotion-expressing faces, then return the dominant emotion from the expressive image. This saves me time in abstracting vital information from the user's data instead of relying on manual operations. The package's limitations are that the training dataset varied in quality and representativeness, and lack of contextual information provided during training [9].

5.4. Roundcubemail

Roundcube is an open-source Internet Message Access Protocol (IMAP) client written in PHP with support for external IMAP/SMTP servers. It allows for customisation at every level of its architecture with a technology stack (PHP, JS, HTML/CSS) that is widely understood and provides extensive support for custom plugins [8]. It serves as the user-facing email platform and allows for the addition of my custom "Mood Detector" feature in the email composition toolbar, from which the facial detection and animation workflow is triggered. The ability to run Roundcube locally was essential to my development because it allowed for seamless injection of new user interface components and portrayal of a real-world use case, whilst alternative platforms (like ProtonMail) required altering core files within extremely large codebases.

5.5. Delanuay Triangulation

In computational geometry, a Delanuay triangulation is a set of triangles made from a discrete set of vertices so that no vertex lies inside the circumcircle of any triangle in the set. Essentially, a method of triangles used to optimally map out a set of vertices/points on a 2d plane [19], resulting in the creation of a mesh useful for image processing. I completed the Delaunay triangulation of the average landmark positions between both faces, creating triangles that are "well-shaped" and avoid skinny/sliver triangles, leading to smoother warps.

5.6. OpenCV

OpenCV is an open-source computer vision and image processing library [20]. It is used in my application for image transformation and processing, and was chosen for being the standard in computer vision projects.

5.7. Gmail IMAP/SMTP

Users can send and receive emails through Gmail, and it also handles sign-in authentication, utilising its services ensures system security through trusted means.

5.8. PIL

It was used to load and resize the two images to the same size whilst maintaining their aspect ratio, so pixel correspondence could be calculated accurately. This is needed because PIL images can also be easily converted to NumPy arrays for OpenCV processing.

5.9. Imageio

A Python library that makes it easy to read and write a range of image data, It was used to assemble generated frames in the animation process and write them in the desired format as '.gif' files [21].

These tools collectively enable smooth integration from webcam capture and facial expression analysis to animated GIF generation and seamless email embedding, addressing both the technical requirements and user experience needs of this project.

6. Architecture and Design

To successfully create my application, I have developed a modular system in which multiple standalone but independent components communicate to create a seamless workflow. My system comprises three main components that interact using the Model-View-Controller pattern:

- The Roundcube email integration (user frontend),
- The backend Flask server (image processing engine and middleware)
- The camera interface is the primary point of data entry and user interactions.

Combined, these different aspects enable the users to capture their facial expressions, be informed of their detected mood, create a custom animated GIF and embed the result directly with their email message. These core components will aid in making digital correspondence more emotionally expressive and natural.

The next section of work visually depicts the overarching workflow for my system, illustrating the user and data flow, along with component interactions.

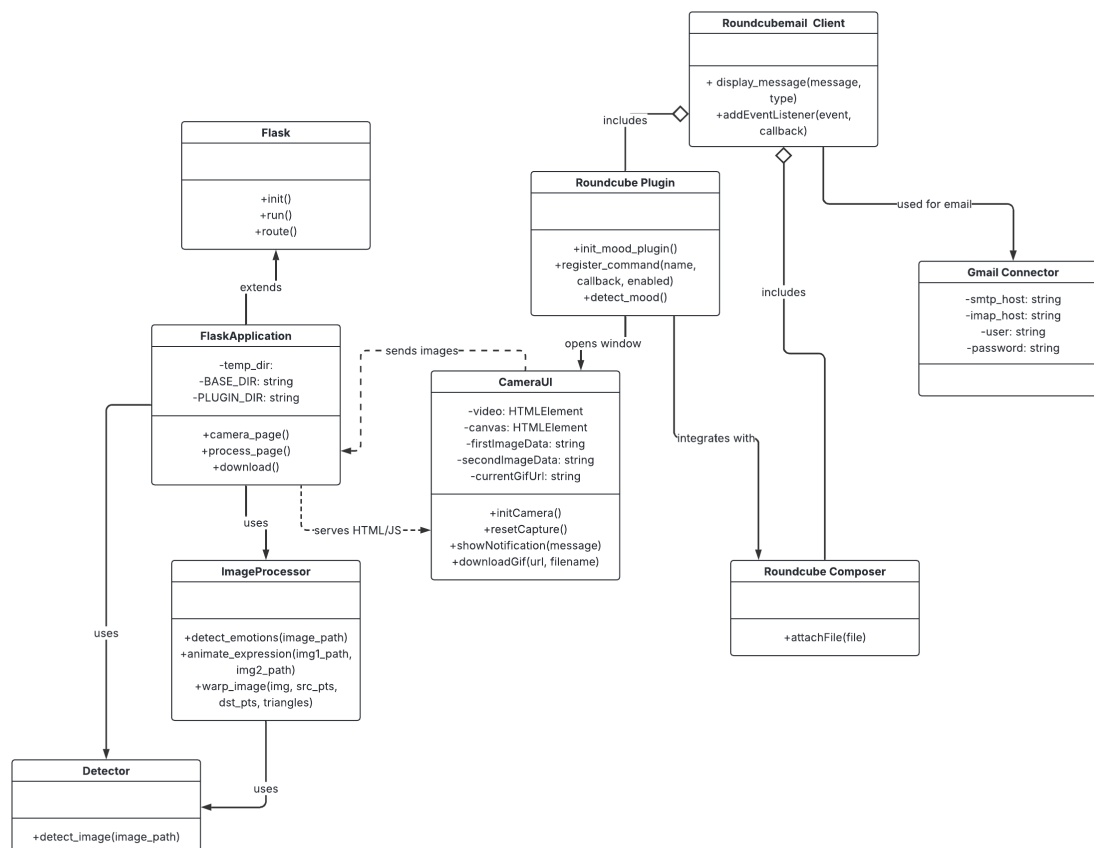


Figure 1. System UML Diagram

See Figure 1; The UML diagram showcases the necessary classes and their relationships.

6.1. UML Core Components

6.1.1. Camera UI

Central to my system is the camera user interface 'CameraUI', it is responsible for capturing the facial expressions, issuing processing requests and previewing the resulting animations. This is essentially the view aspect of my system, as it handles the format in which data is presented to users without playing a part in data processing. Its key functions are to handle webcam initialisation and access, guide users through the two-step image capture process by displaying relevant notifications, prompts, detected emotion in the expression image back to the user, a preview of the captured expression image and animated GIF. It also provides user controls for capture, regeneration, and downloading the animation onto the user's device. It interacts directly with the 'FlaskApplication' controller via HTTP requests, triggering routes corresponding to image processing and download.

6.1.2. Plugin

The 'RoundcubePlugin' or 'mood_plugin' as defined in my code, integrates the mood detection functionality directly into the frontend email client, in this case, Roundcube. It grants the user access to the backend powerhouse by adding the "Detect Mood" button to the email composition window, allowing for the camera interface to be opened in a pop-up window (which it hosts). It also receives the animation data via the postMessage API. It is built onto 'RoundCubeComposer', a section within my host application 'Roundcubemail Client'.

6.1.3. Flask Application

My 'FlaskApplication' is the processing engine of my system, acting as the modal aspect. Inheriting from 'Flask', it is responsible for all computational procedures, which include facial analysis, emotion detection and animation creation.

In addition, it is the storage system for data created during runtime, concurrently acting as a navigator to system files and a router to my '.js' controller files.

6.1.4. Image Processor

'ImageProcessor' encapsulates the emotion analysis, core facial landmark detection and animation process; it relies on the 'Detector' module for basic image analysis.

6.1.5. Gmail connector

Abstracts the required IMAP/SMTP settings for external mail server communication.

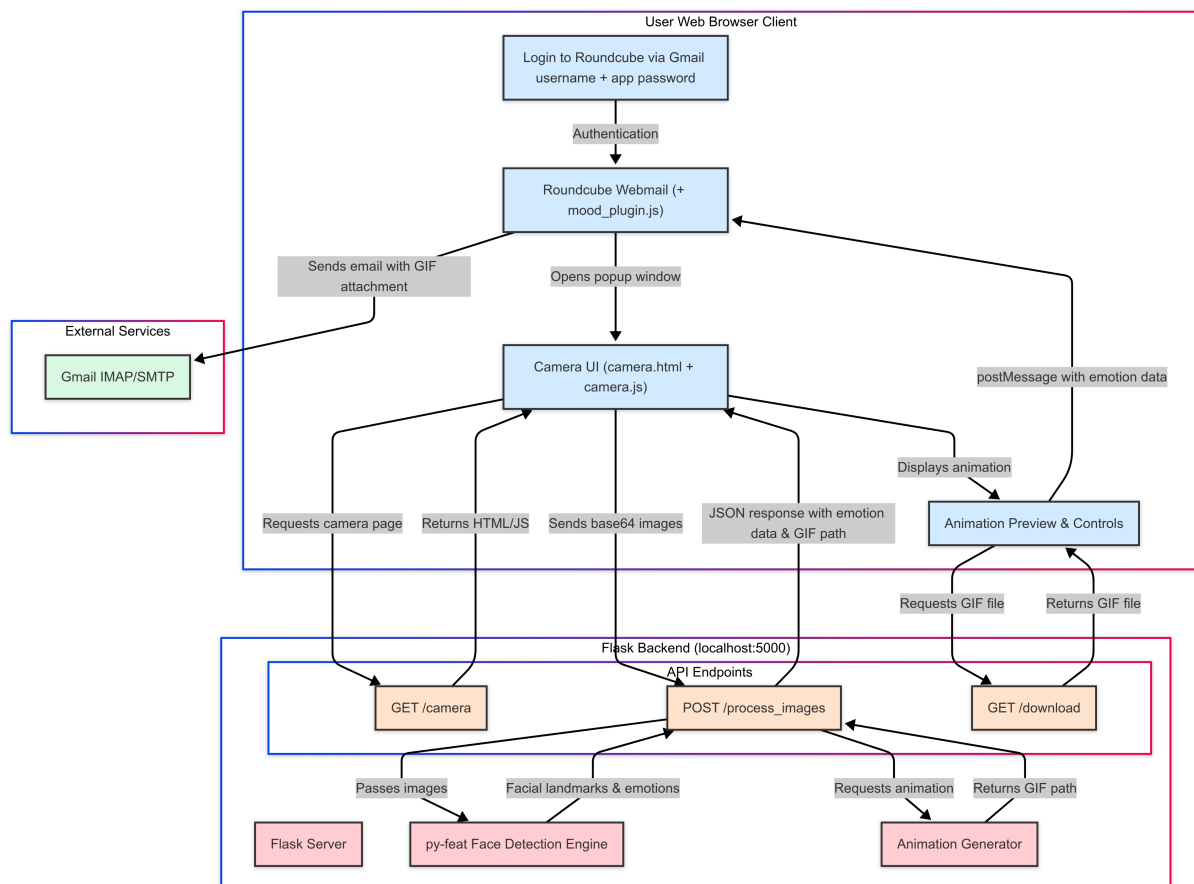


Figure 2. System Architecture Diagram

See Figure 2; Illustrated here is a detailed system architecture diagram that demonstrates the workflow of my project by compartmentalising components into the front-end and back-end and email service, respectively.

6.2. Architecture Assumptions and workflow Summary

For my system to run, these three assumptions are made:

- The user has configured their email service for IMAP/SMTP
- The backend Flask server is running
- The frontend Roundcube application is running

The application begins in the user's web browser block, when a client logs into their email account on the standalone Roundcube web client through localhost with their personal username and generated App password. At this stage, authentication occurs using the account host IMAP. Upon successful login, on the compose page they will find a “detect mood” button injected by the mood_plugin PHP file and its corresponding JavaScript handler.

When clicked, it triggers the camera interface that chronologically prompts the user to capture two images (neutral and expressive), both of which are converted to base64 [23], encrypted using Transport Layer Security (TLS) and transferred to the Flask backend via HTTPS POST. The backend conducts analysis and processing of these images, producing a morphing GIF and returning the emotion label and GIF path to the front-end, where users can preview their captured expression and animation before deciding on what to do with the output. Keeping the image data transient, enabling HTTPS on all sensitive data sent between the web client and the Flask server, enforcing user authentication at the email client, and protecting backend logic behind controlled API endpoints, system security is maintained by ensuring privacy of data transmission using trusted means of origin verification and encryption.

Secure authentication to the email service takes place via Gmail's IMAP and SMTP protocols using user-provided App passwords. This allows Roundcube, running, to send and receive emails on the user's behalf.

6.3. Core Components – linked to UML

- CameraUI: As the primary point of data entry, its separation from the backend logic as a standalone aspect enables future modular improvements, for the benefit of the front-facing user, without risking tampering or corrupting critical program logic.
- Roundcube Plugin: This is the mediator between my Roundcube frontend and my system's custom interface, enabling seamless end-user access.
- Flask Application: acts as the logic controller, a temporary storage manager, and HTTPS certificate issuer, ensuring data is kept only as long as necessary and shared with verified sources, in compliance with common production practices and GDPR principals [27].
- Animation generator and Detection Engine: Image analysis support an extensible pipeline-based architecture. This means that alternative

models can be substituted for alternative models based on user preference or future developments.

- Mail Connector: By isolating the external mail platform communication, users are granted choice in the mail provider they would like to configure the Roundcube mail client, magnifying the utility of my application.

6.4. Rationale and Design Choices

As previously mentioned, the core architectural pattern of my system is the Model-View-Controller, chosen for its ability to separate consumer logic from backend processes. It allows me modular testing, modification and maintenance whilst protecting sensitive information from unnecessary exposure. The use of layered security through TLS/HTTPS, temporary storage, authentication and origin verification creates a private tunnel in which the component of my system can safely interact without risk of man-in-the-middle attacks [25].

6.5. Limitations of Design

As it stands, my design is not without flaws; the tight coupling of my plugin with Roundcube means refactoring the application for a different email service may prove to be difficult. With higher quality images, base64 encoding may be unsuitable as it increases the file size by approximately 1/3 of the original [23]. However, this is largely dependent on various factors like the encoded string's length. These issues point to scalability being a possible hurdle in the future but rely on how the application is adopted in different environments.

7. Methodology

The methods of development chosen were to result in the most functional solution within a limited period, whilst accounting for the possibility of requirement changes and challenges and tailoring my application to the research feedback collected. With this strategy, I can periodically create testable prototypes, ensuring a working version by the deadline.

7.1. Survey Design

To gain insight into the common misinterpretations and connotations of non-textual inclusions in conversations and aid in effectively creating my application. I surveyed 31 employees of different companies, using a varied format of multiple-choice, Likert-scale and checkbox questions, to explore how non-textual elements influence tone and clarity in personal versus professional conversations. The survey was split into two sets of questions, the first focused on the existing solutions to the problem, and the second set focused on my application's solution.

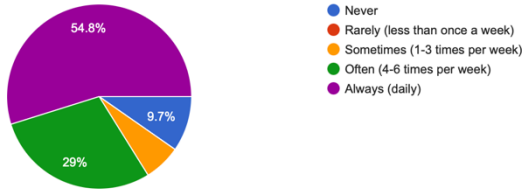
The data gathered allowed me to make factual inferences about the cause of the stigma around the topic and address possible reasons why using these elements to prevent conflict arising from a lack of emotional context in work emails is not as prevalent as expected. To gain feedback on gaps in the existing solutions my system could fix, and the critical consumer opinion of the proposed solution.

7.1.1. Set 1

How often do you include emojis in your personal text messages?

31 responses

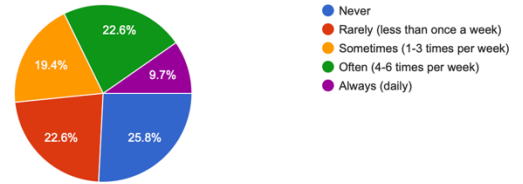
[Copy chart](#)



How often do you include GIFs in your personal conversations (e.g. iMessage, WhatsApp)?

31 responses

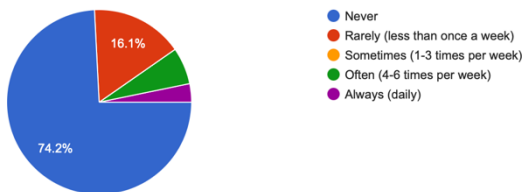
[Copy chart](#)



How often do you include emojis in professional conversations (e.g. email, Teams)?

31 responses

[Copy chart](#)



How often do you include GIFs in professional conversations (e.g. email, Teams)?

31 responses

[Copy chart](#)

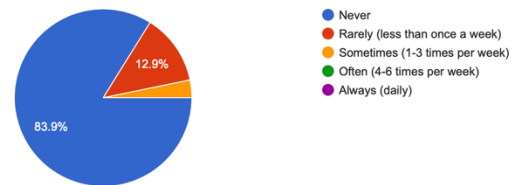


Figure 4. Emoji use frequency for personal setting

Figure 3. GIF use frequency for professional setting

When respondents were asked how often they included emojis in personal conversations, the vast majority (54.8 percent) stated “Always”, whereas when asked about the frequency of GIF use in personal conversations, results were more split but showed that most people (25.8 percent) stated “Never”. Similarly, presented with the same question but for a professional setting, respondents answered 74.2 percent for “Never” and 16.1 percent for “Rarely” with emojis and 83.9 percent for “Never” and 12.9 percent for “Rarely” with GIFs. Now, comparing the two findings, we can deduce that even with how well the use of emojis has become habituated in personal conversations over GIFs it struggles equally as much to find its place in a professional work environment (see Figures 3 and 4).

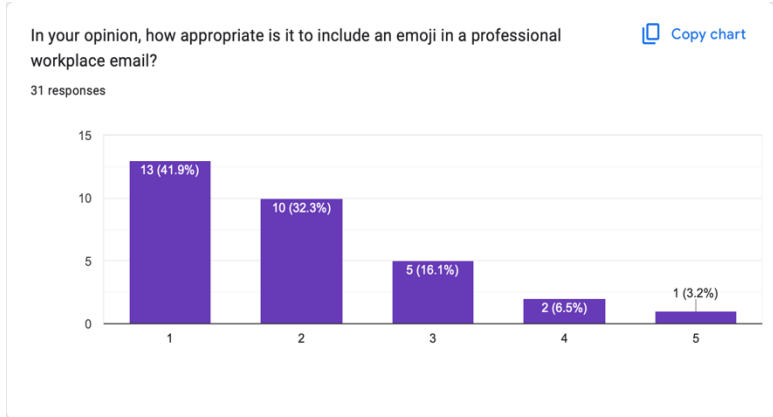


Figure 6. Opinion of emoji in workplace setting



Figure 5. Opinion of GIF in workplace setting

The next questions were to gauge participants' opinions on the appropriateness of emojis and GIFs in a formal context. A side-by-side comparison of the results shows that participants find both methods of conveying emotion and tone to be "Very inappropriate", with 41.9 percent and 45.2 percent respectively, the rest of the responses followed a downtrend from this point indicating that the consensus amongst the sample population is that they shouldn't be included at all (see Figures 5 and 6).

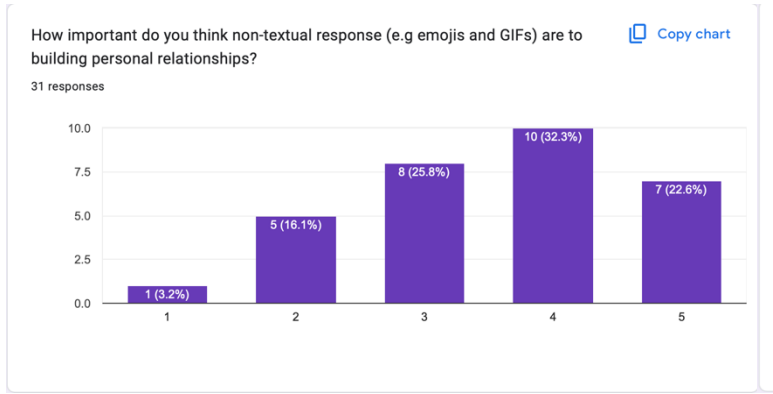


Figure 8. Importance of non-textual responses for personal relationships.

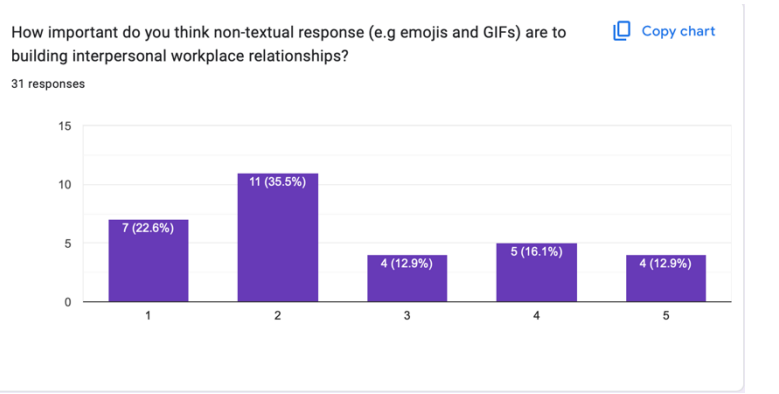


Figure 7. Importance of non-textual responses on workspace relationships

However, when asked about the importance of these tools for building both personal and workplace relationships, ranging from "Unimportant" to "Very important", a more positive response was received for personal relationships with 54.9 percent of respondents ranking it "Important" to "Very Important". For interpersonal workplace relationships, though, a mixed response was given (see Figures 7 and 8).

7.1.2. Set 2

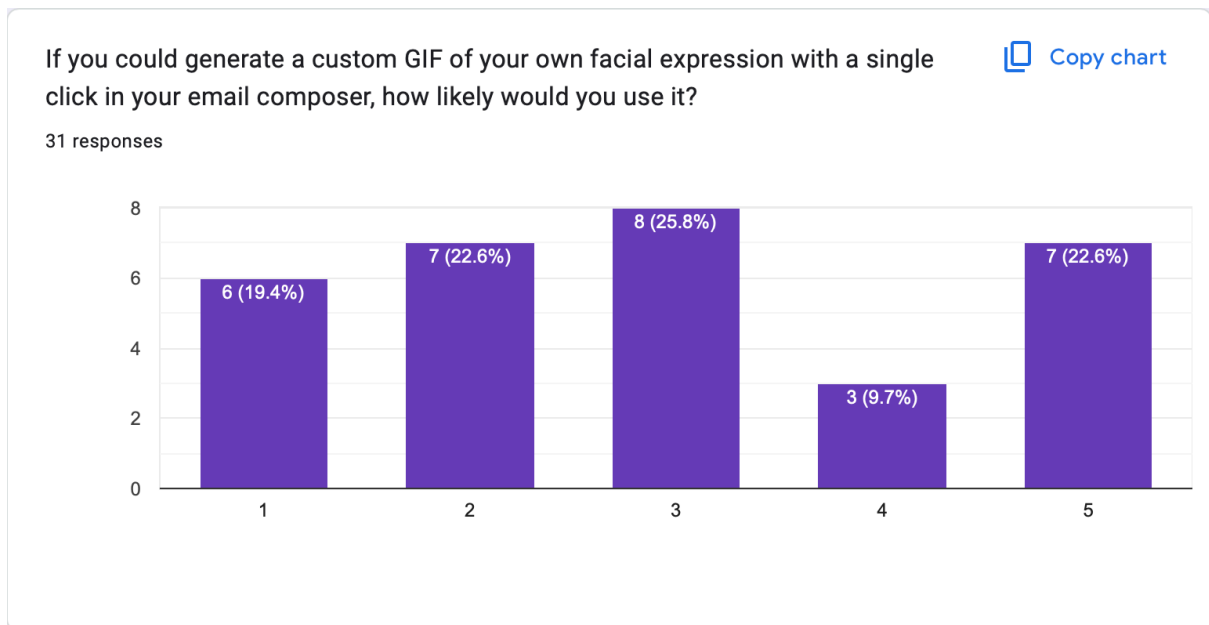


Figure 9. Opinions of a Mood based GIF animator

Results of this question indicate most responder 25.8 percent do not have a strong partiality to the system, if we focus on both ends of the scale however, we see that more people (22.6 percent) state that they are “Very likely” to use the application if made readily available versus 19.4 percent stating they are “Very unlikely” to do so. The impartiality may be attributed to the negative connotations people already have of using GIFs in their conversations (see Figure 9).

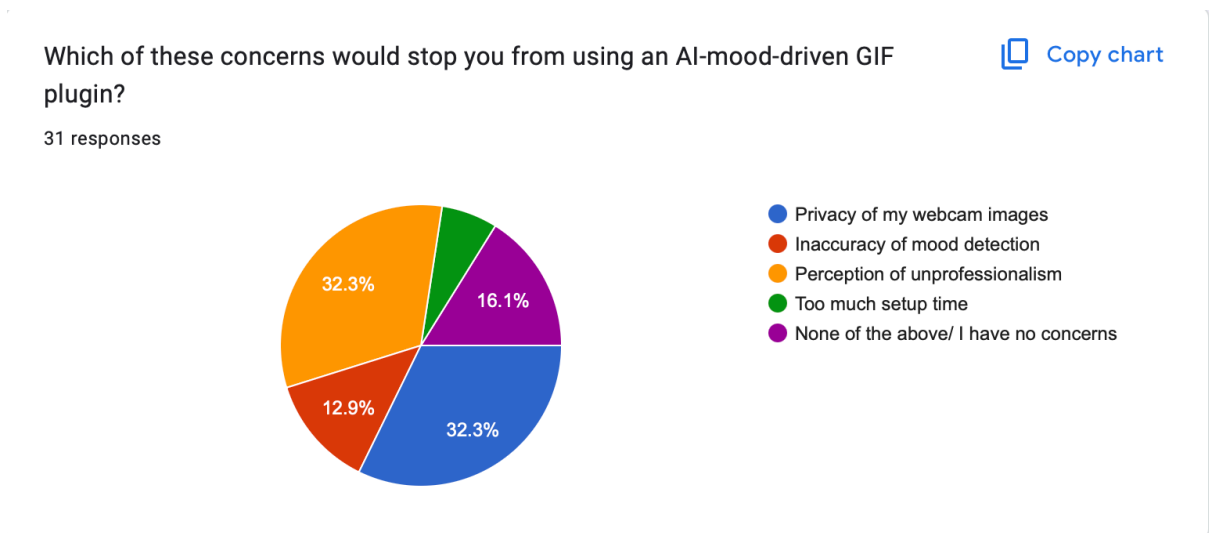


Figure 10. Limitations of using Mood Based Gif Generation

The data illustrates to us the main concerns of the application: “Privacy of webcam images” and “Perception of unprofessionalism”, with 32.3 percent of participants picking either of these options (see Figure 10).

7.1.3. Summary of survey findings

To summarise, most participants believe that including GIFs and emojis is more suited for personal conversations rather than professional. They understood the importance of using them to build stronger informal relationships yet fail to recognise how these same tools could be used to build professional relationships, as previous studies show. This is mostly due to the fear that they may be perceived as unprofessional but can also be associated with the lack of appropriate user choice when it comes to these tools.

7.1.4. Limitations of research and future improvements

There are some limitations to my research, which could hinder the reliability of the data collected. Firstly, the small sample size means limited representation of different job sectors, which could lead to biased results by overrepresentation or exclusion of a specific industries. Additionally, the lack of categorising my sample makes it more difficult to generalise results collected into the larger population

Given the opportunity to carry out this research again, I would focus on gathering a larger sample the grouping them based on age, job sector and work environment.

7.1.5. Survey findings conclusion

Increased focus should be put on creating an application that will easily integrate into existing workflows, whilst offering users a choice in the type of animations they can create. If my program is to find success, I must device a work-around for the current perceptions of non-textual context and enforce security measure to ensure data privacy, most importantly of images my application captures.

7.2. System Design

My application was developed using the personal Agile methodology, a project management strategy that breaks down projects into several manageable development cycles - Sprints (smaller phases of development) following.



Figure 11. Agile development cycle [26]

Through this, I was able to focus my efforts on the areas that needed improvement post-sprint review.

The agile methodology boasted many advantages, which I greatly benefited from. Compared to other commonly used development models, e.g., the Waterfall Model. The agile model enables 'agility' as the name suggests, in core requirements during production. This is facilitated by the incorporation of user feedback at the end of each cycle to ensure that the final product is as close to expectations as possible. Post review, further planning can be made on improvements required in the system for the next sprint to better comply with the final product requirements (see Figure 11). On the other hand, the Waterfall model, which is largely regarded as outdated by developers, would see the entire system built based on the gathered initial requirements, then reviewed once completed, changes required pose the risk of requiring a complete rebuild of the application. With the fluidity of program development and time constraints, an approach such as that is very time-consuming and may result in fruitless labour [26].

Development consisted of three core sprints, individually focusing on different components of my system that needed improvements.

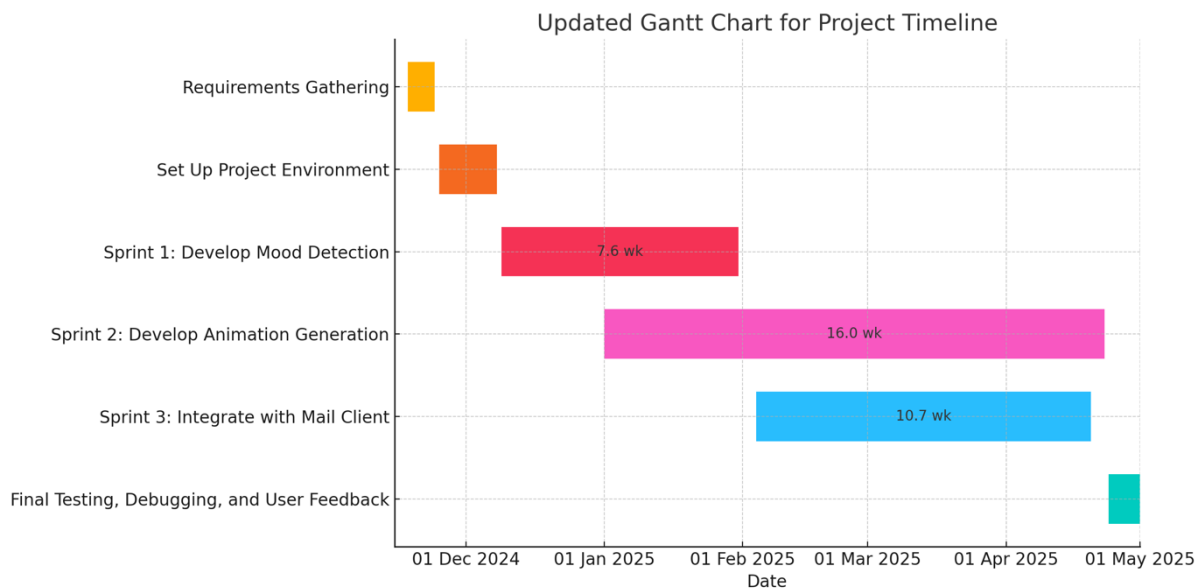


Figure 12. Project Timeline

The use of parallel sprints as a solo developer is unconventional, but due to the challenges faced, I decided the adoption of this approach was necessary to ensure progression in every aspect of my system. Had I not taken up this strategy, I would have certainly missed the deadline for my system (see Figure 12).

7.2.1. Sprint 1 (8 weeks)

During this period, a prototype of my system was made via means of a dummy application. This was to model the primary structure of the final product. Creation of this followed a breadth-first approach, encompassing the core components my system would require. The goal at this stage was to create my image capture interface for data collecting. I also attempted to implement the image processing logic for mood detection and animation. To do this, a simple HTML email platform was made to emulate the system workflow, it integrated a button to open the user's device camera and capture an image of their face, then send it to my backend Flask server for processing.

Reviewing Sprint 1: By the end of this sprint, I couldn't produce a completely working application. The only functional component was the image capture. It was flawed in many aspects because at this moment, my comprehension of the packages and libraries required for data processing was inadequate. Nonetheless, it visualised the system architecture that my final program would need to comply with.

7.2.2. Sprint 2 (16 weeks)

This segment of development targeted the depth of my system, incorporating user expectations and essential system requirements. This sprint was research-heavy, requiring delving into the computational methods of detecting facial expressions and the art of image morphing. I tested different methods of implementing the programmatic aspects within my system. Alternative methods tested include automated image capture on plugin trigger, single image capture interpolation to animated GIF, and directly integrating my program into the Roundcube source code.

Reviewing Sprint 2: This sprint addressed concerns made by participants in the research survey regarding my implementation, such as the privacy of webcam images. By putting systems in place to encode captured images to base64, then encrypt using the Transport Layer Security (TLS) protocol or SSL as it was previously known, communication is secured using the asymmetric public key infrastructure, meaning users can rest assured their data is protected from outside interference [27].

7.2.3. Sprint 3 (11 weeks)

The final sprint focused on the integration of my software into an email service. There were two methods I used in this instance. The first was creating a custom SMTP mail server using Mailhog while developing on the previous HTML platform I had built, but due to the purpose of my program and my intention for it to be usable in production, simply using a fake SMTP email testing tool wouldn't meet the benchmark.

The next approach was to find an open-source email service that allowed for custom plugins. Through research, I chose Roundcube, creating the plugin required modifying the source code by adding custom plugin files that would enable communication between the front email platform and my backend processor (Appendix E).

Reviewing Sprint 2: Since my project wasn't centred on developing an email platform, but instead on adding a layer of functionality. I reviewed the utility of creating a custom email platform and concluded that using an existing email service would be more time-effective and presents the opportunity for publication in the future. This approach allowed me to leverage Roundcube's existing UI while meeting the real-world requirement of sending genuine emails rather than MailHog fakes.

8. Implementation

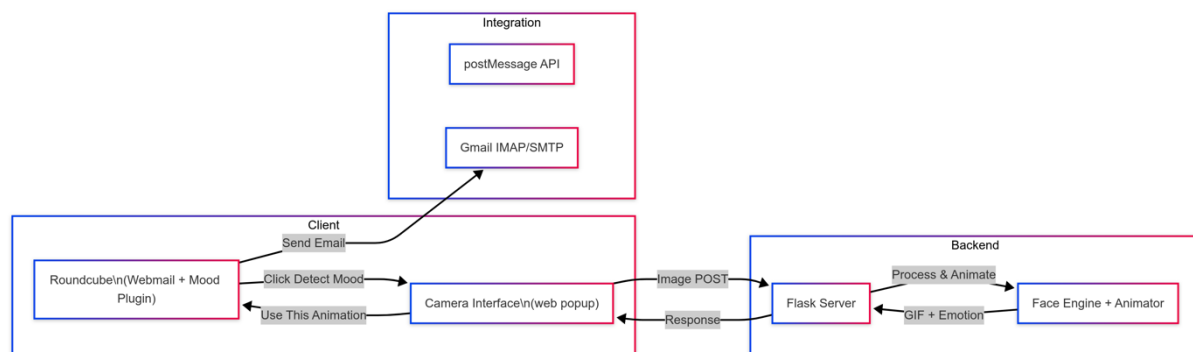


Figure 13. Summarised System Architecture Diagram

8.1. System Development

Figure 13. shows a summarised system architecture diagram of my 3-way system, which demonstrates component interactions between the backend, client, and external email service.

To develop my system, I deemed it a necessity to integrate it into an email service to truly showcase the real-world application, a conclusion arrived at by experimenting with an existing application and a custom email service.

8.1.1. Configuration

The Roundcube Webmail suite must first be cloned from its source GitHub repository and following its 'README' and 'Install' documents, it can be set up for local development. It requires configuration with a local database, such as MySQL, details of this must be added to the 'config.inc.php' file. Functionality requirements include an IMAP, HTTP and SMTP server which can be designated to an external email service, in this case Gmail [29]. Once complete, the user's account username and App password can be used to handle authentication on the platform, which grants access to their live email feed. My plugin files can then be injected directly into the application's source code, within the plugins folder and referenced as a plugin in 'config.inc.php'. This will allow access to the custom Camera interface, which in turn communicates with my backend Flask server (see Appendix E).

Based on preference, this step can be changed to work with any other email service that provides support, a desktop-like user interface that is familiar to the users, as it follows the general structure of email platforms.

8.1.2. Mood Detection

Mood detection was programmed using the provided Py-Feat detector class. After instantiating, Py-Feat will automatically download pre-trained models suitable for an array of vision-based tasks. By default, Py-Feat uses Residual Masking Network (ResMaskNet) for emotion detection [30], a Convolutional Neural Network (CNN) model that combines deep residual networks with masking blocks, which allows the model to isolate key local regions of interest to refine its feature mapping. It is trained on the FER2013 and VEMO datasets, which contain (28709 images for training, 3859 for validation and 3589 for testing) [31]. Based on research, accuracy is dependent on the testing dataset, and the topic of emotion classification is still quite subjective, official documentation, and the most common data points to an average accuracy of 74% [31], [32]. However, it is important to note that developers of Py-Feat carried out their testing on a dataset of 500 random images from the AffectNet dataset and documented an average F1 score of 0.55 or 55% [30].

Passing an image as a parameter to:

```
df = pyfeat_detector.detect_image(image_path)
```

will return a dataframe of the resulting image analysis (see example below).



Figure 14. Stock image for analysis

Figure 14. is an example stock image, similar to what my mood detection function could receive. Assuming a face is detected in the image, it will return a dataframe containing 173 columns of all the information Py-Feat extracts from the image, the majority of which is unnecessary to this process, e.g., 'FaceRectY' and 'FaceRectX' (see Figure 15).

```
INFO:__main__:Result: FaceRectX FaceRectY FaceRectWidth FaceRectHeight ... surprise neutral input frame
0 287.234711 29.759499 82.050087 119.433362 ... 0.340842 0.001369 images.jpg 0
[1 rows x 173 columns]
```

Figure 15. Result of running the Py-Feat detector on an image

To then target the feature we are looking for, we run a Pandas command for indexing and aggregation, which will target the 'emotions' sub-DataFrame:

```
dominant_emotion = df.emotions.iloc[0].idxmax()
```

The output of which is:

```
INFO:__main__:Detected emotion: happiness
```

Figure 16. Outputted emotion label

This emotion label can then be packaged and returned to the front-end client as a notification, providing awareness of the mood they are currently portraying, enabling adjustments if necessary, before the animation process.

8.1.3. Animation Generation

The process of creating an animated GIF from a neutral image to an expressive image was laborious, but possible because of an online resource [11]. The 'animate_expression()' function implements a face morphing pipeline. It was completed using the following technology stack:

- Image resizing with PIL: This loads both images into the function, then resizes the second image based on the first image's dimension, maintaining their aspect ratio to reduce detail loss via the LANCZO filter. The resized images are then saved to the temporary directory because Py-Feat requires file paths to landmark detection.
- Landmark detection: Using the packaged MobileFaceNet model with Py-Feat, landmark results can be extracted from the processed images, which locates and maps out 68 key facial points, e.g., eyes, nose, mouth and jawline with a 99.95% accuracy [33]. If detection landmark detection fails, meaning a face wasn't detected in the image, the process is aborted.

- Delaunay triangulation via SciPy:

```
corners = np.array([[0,0],[W,0],[W,H],[0,H]])
pts1 = np.vstack([lm1, corners])
pts2 = np.vstack([lm2, corners])

mean_pts = (pts1 + pts2) / 2
tri = Delaunay(mean_pts)
```

```
triangles = tri.simplices
```

Using Numpy and SciPy's Delaunay function, we can easily subdivide the set of facial landmarks with four image corners to ensure full-frame coverage into a mesh of non-overlapping triangles, based on the average shape to maximise the minimum angle in each triangle and avoid the occurrence of skinny triangles and allow for visually smooth warps.

The 'warp_image()' function was created with reference to a tutorial by Azmarie Wang [11], and a GeekfofGeeks guide [34], with the aid of OpenAI's ChatGPT model as a debugger [35]. Image morphing is a process of geometric transformation, requiring multiple subprocesses that combine to perform a transformation matrix.

- Affine Warping of Triangles: For each triangle in the Delaunay mesh, OpenCV's cv2 performs an affine transformation from the source to the destination triangle, then warps the corresponding image patch. This allows for precise mapping of triangular regions that allows for the geometric "morph effect" [36].
- Cross-Dissolve Blending: To blend the two warped images, a cross-dissolve is performed to smoothly transition pixel intensities.
- Frame sequencing: Multiple intermediate frames are generated between the two facial expressions, neutral and expressive, and a reverse sequence is applied to give a "ping-pong" effect. These frames are stitched together with imageio's 'mimsave()' with the parameter "0" to produce an infinitely loop GIF.

8.1.4. Integration

The pop-up camera interface is made up of the standard HTML5, CSS, and JavaScript used for webpage design. With these, I created a clean and interactive front-end, where users view the animation process in action. It provides a camera preview of each capture, post-capture results such as the detected mood and access to the generated GIF (see Appendix F).

Essentially, functional features that allow interaction with the backend Flask server via Fetch API endpoints like POST [37], handled by the 'camera.js' file. These enable core user abilities, such as buttons with added event listeners that guide the users through steps within 'camera.html':

```
<button id="snap">Capture First Expression</button>
```

; notifications that provide status updates:

```
status.textContent = 'Processing...';
```

; and process regeneration that resets the system workflow in 'camera.js':

```
document.getElementById('regenerate').onclick = () => {  
    resetCapture();  
};
```

Finally, the camera UI is a means to throw system errors at the user in an understandable and "prettier" format using notifications.

8.1.5. Data Encryption

(See Appendix G)

Data encryption between the front and backend is provided by HTTPS, which uses SSL/TLS protocols. It employs an asymmetric public-private key pair to secure all communications over Transmission Control Protocol (TCP). Following a programmatic approach with self-signed SSL certificates, image data, mood detection results and generated GIF file paths are encrypted in transit.

Firstly, a 2048-bit RSA private key is generated and stored locally. Next, a Certificate Signing Request (CSR) is created in code with the issuer and subject details (i.e., country, state, organisation and common name), then self-signed using SHA-256 hash algorithm, proving key ownership, to produce a X.509 certificate that has a life-span of one year. The ('cert.pem') and private key ('key.pem') are loaded into a Python 'ssl.SSLContext' object that is used to serve HTTPS traffic [38].

The Flask server uses its private key in the TLS user-server handshake process, where the server presents its certificate to the client; the client verifies its

authenticity, then they negotiate cipher settings, and establish a shared session key for communication there on.

The HTTPS layer uses the session key to decrypt and encrypt data before handing it to the receiver. This structure guarantees that the client is talking to the intended server, and if an attacker intercepts HTTPS traffic without the session key, they cannot decrypt the sensitive data exchanged between my plugin and the backend Flask server [39]. The process is automated on start-up to reduce manual setup, but includes added checks to confirm and use a certificate if it is already provided by the user.

Currently, as this is a self-signed certificate, browsers will flag it as untrusted unless manually added to the list of trusted certificates in the user's system. The alternative is sending our CSR to a Certificate Authority (CA) to sign, often requiring an underlying paid cloud service or payment to the CA authority [39], [40]. In large-scale production, this would probably be more suited as it provides strong encryption and a high level of trust.

8.2. Optimisation and Performance

Many considerations were made to make my application an iterative process that enables the frontend user to run the program uninterrupted.

- **Memory Management:**

The implementation of my system considers file storage; the live capture images and created GIFs are saved in a temporary directory created on startup in the Flask backend:

```
temp_dir = tempfile.mkdtemp()
```

On session termination, these files are wiped from the user's disk, an essential component of memory management required for my system to run efficiently. This method boasts a variety of benefits, starting with preventing memory leaks by ensuring processed images don't accumulate, allowing the application to run much longer without memory degradation and finally allowing for the processing of multiple high-resolution facial images.

- **Disk Space Optimisation:**

The temporary file storage system prevents the uncapped growth enabled by using a fixed directory. This mitigates the risk of excessive accumulation of temporary files, which take up large amounts of disk space, weaken system performance, possibly slowing file retrieval time, and pose a greater threat of disk fragmentation. In combination, these factors would result in a sluggish user experience, but scrubbing the temporary files at the end of each usage ensures performance consistency.

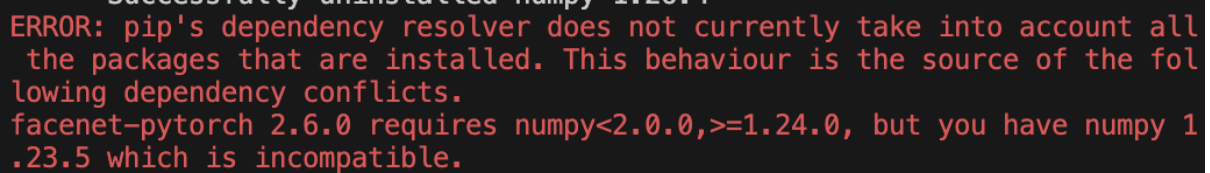
- **File Size Optimisation:**

To comply with Roundcube's 2mb file attachment size, the generated GIF's file size is checked; if it is over the limit, a 'while' loop is used to iteratively check and reduce the number of frames in the GIF to a minimum of five, if that doesn't work then the resolution of the GIF is reduced by 10 percent until the target is met.

8.3. Challenges

8.3.1. FER

In the original proposal of my system, I had concluded that I would use the Python FER library for emotion detection in conjunction with Py-Feat for interpolating static images into mood-based animation. In practice, this proved to be impossible because the libraries had conflicting package requirements at every turn. For example, both packages required different versions of numpy simultaneously, which is impossible, even using a virtual environment, as the code would be contained in the same workspace (see Figures 18).



```
ERROR: pip's dependency resolver does not currently take into account all
the packages that are installed. This behaviour is the source of the fol
lowing dependency conflicts.
facenet-pytorch 2.6.0 requires numpy<2.0.0,>=1.24.0, but you have numpy 1
.23.5 which is incompatible.
```

Figure 17. FER & Py-Feat dependency conflict

To resolve this, I made the executive decision to remove any usage of FER and switch completely to Py-feat for the mood-detection aspect. This proved to be advantageous because Py-feat offers more advanced capabilities, such as facial landmarks and Action Units, which later would be beneficial to the animation process. Supported by the fact that Py-feat offers state-of-the-art models like OpenCV with a higher accuracy in emotion detection, this was the most sensible decision.

8.3.2. ProtonMail

A key step in integrating my system was finding a suitable open-source email client to host my system. Initial research led me to ProtonMail web client, a monorepo project managed with Yarn [41]. However, this platform proved to be extremely problematic and introduced numerous development hurdles. Following the "README", I successfully cloned and configured the program on my local computer, but I was immediately faced with an ironclad login page. I attempted to bypass it by referring to the documentation for developer login instructions, which was disappointingly inept (see Appendix A), then I tried to consult the developers of the program and they too

seemed stubbed by the problems I was facing, blaming it on server issues, with a developer stating “it’s not easy to do” (see Appendix B). I also tried to use my login details but was met with what seemed to be a common issue developers faced when working with the program (see Appendix C).

With limited online resources and no customer support, I searched through the over 220,000 file codebase for the authentication modules and tried to hard-code developer credentials, but this approach also failed.

Finally, I implemented a mocking approach; a “mock-API” server to intercept and handle authentication calls intended for ProtonMail’s backend. This introduced further complications with the introduction of an additional problem, an SSL certificate error when connecting to the mock server (see Appendix D). I attempted to resolve these by self-signing certificates, which only compounded the problems. At this point, I decided the platform was more work than it’s worth and began evaluating alternative platforms.

8.3.3. Animation

The main challenge in my development was the animation process; multiple methods of animation were attempted. Using the ‘animate_face()’ function Py-Feat provides, I attempted to morph a singular facial expression into another by interpolating between AU intensities, however, this was unsuccessful because this method uses a default drawing rather than the given parameter image and continuously resulted in the following animated GIF (see Figure 19.) to my understanding this meant the function ability was solely for demonstration and could not be overlayed onto actual images.

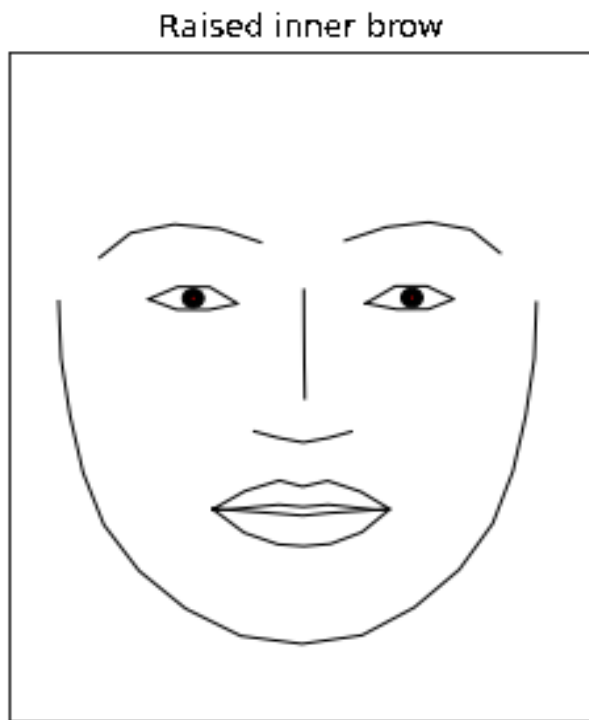


Figure 18. Generated `animate_face()` GIF [9]

The next approach was the `'interpolate_au()'`, which by description “will generate intermediate AU intensity values between two arrays in a manner that creates graceful animations” – this too worked by drawing lines to illustrate the animation.

Realising that Py-Feat's functions would not work, I utilised the landmark detection that this package provides and, using the collected data, I tried to manipulate it to match that of the expected opposite emotion then create a new image using the resulting landmarks positioning; instead, this created a layer of plotting on the given stock image which wasn't the intended purpose (see Figure 20).



Figure 19. Output of Py-Feat Animator

As none of these methods worked, I developed the Delaunay triangulation method, containing certain aspects I learned from my previous attempts (e.g., interpolation), which helped to develop a function that worked as intended and was used.

9. Testing

9.1. Testing Strategies:

9.1.1. API Testing/ Error handling:

Postman was used to carry out testing of my Flask server API endpoints:

Test 1 - Pop-up Camera UI:

- Method: GET
- URL: <https://localhost:5000/camera>
- Expected Response: The expected response for this test case was a status code of '200 OK' and the HTML content of the camera page.
- Actual response: "200 OK" and a preview of my camera webpage (see Appendix H).

Test 2 – Image Processing:

To allow for image processing to take place via Postman, I created an image-base64 converter, gave it 2 stock images, then passed the base64 encoding to the JSON body of the post request because my processing engines require image data to compute on.

- Method: POST
- URL: https://localhost:5000/process_images
- Expected Response: Status code "200 OK", the dominant emotion detected in the expression image, and the GIF path to the created animation.
- Actual response: "200 OK", "dominant_emotion: happiness" and the local GIF path (see Appendix I).

Test 3 – Animation/Download:

By testing the '/download' endpoint, I could confirm that the result of my application is an animated GIF, done by giving the HTTPS request the file path to the GIF generated in the previous step.

- Method: GET
- URL: https://localhost:5000/download?gif_path=/var/folders/9h/3x32k78j611450bq0z6qdz9m0000gn/T/tmpplal8tu/face_morph.gif
- Expected Response: Status code "200 OK" and the animated GIF.
- Actual response: "200 OK" and the animated GIF (see Appendix J).

Test 4 – Error handling:

To confirm that my image processing engine verifies the image data that it receives, I tested with two strategies: passing images with empty base64 encoding and images with household items.

- Method: POST
- URL: https://localhost:5000/process_images
- Expected Response: Status code "400 BAD REQUEST" and the error.
- Actual response: "400 BAD REQUEST" and ("error": "No face detected in second image. Please ensure good lighting.") (see Appendix K, L).

9.1.2. Performance testing

Performance testing of the mood detection to animation was done by timing the process 15 times with mixed facial expressions per attempt, as shown below, the average time for image processing was 4.62 seconds. To make the testing fair, inaccurate mood detections and outliers that caused speed in time recording were discarded.

Attempt	Emotion	Time taken
1	Happy	4.64 seconds
2	Anger	4.72 seconds
3	Anger	4.64 seconds
4	Sadness	4.54 seconds
5	Suprise	4.70 seconds
6	Neutral	4.82 seconds
7	Disgust	4.70 seconds
8	Neutral	4.65 seconds
9	Happy	4.60 seconds
10	Anger	4.69 seconds
11	Sadness	4.64 seconds
12	Suprise	4.65 seconds
13	Sadness	4.55 seconds
14	Fear	4.05 seconds
15	Neutral	4.69 seconds
Average		4.62 seconds

9.1.3. Integration testing

Integration testing was carried out by sending an email through the RoundCube mail service with an attached GIF to an external receiver, this was successful (see Appendix M and N)

9.1.4. User Workflow Testing:

To emulate the full sequence of an end user, I manually went through the full user journey, beginning on the Roundcube login page. I used my Gmail account login details – but also carried out basic security testing by using invalid details to ensure prevention of unauthorised access, went to the Compose page, clicked the 'Detect Mood' button, followed the process of mood detection and animation generation, and then proceeded to attach it to an email that was sent off to an external address (see Figure 20).

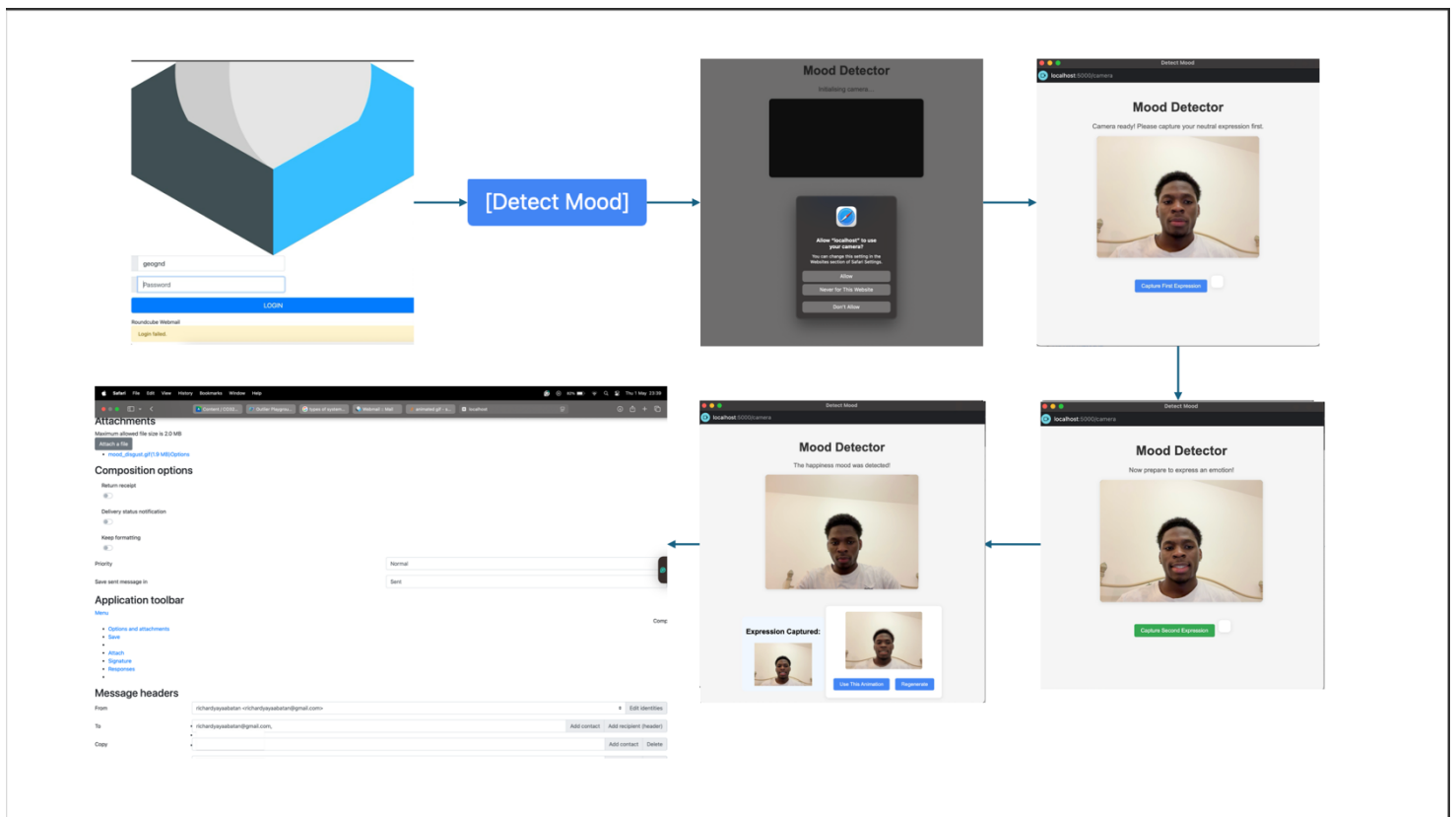


Figure 20. User Workflow Testing

9.2. Limitations of Testing

The weakness of my testing was that I couldn't gather a large number of people to use the application and provide feedback. This meant that user feedback was limited.

10. System Evaluation and Future Improvements

10.1. Evaluate Against the Requirements

The developed system successfully demonstrates the integration of live mood-influenced GIF generation within an email composition workflow. Here, I will be evaluating the success of my application against the aims and objectives:

- Performance: Both mood detection using Py-Feat and GIF generation were completed within the specified 5-second timeframe, confirmed by performance testing, thus meeting the non-functional performance requirement. However, performance may worsen over time due to the base64 encoding used to transmit the image data between the front and backend. The reliance on user system capabilities for said processes also makes performance subjective on the device the application is ran on.
- Usability: The front-end gives users progress updates throughout the process. Users can abort my application at any given time and are provided with the option to regenerate the created animations by capturing new images. The manual workflow of the process was intentionally made manual for user engagement but may deter non-technical users seeking a streamlined application.
- Accuracy: The accuracy performance statistic is subjective on the testing dataset, but most benchmarking of the deep learning model used points to a 74% accuracy of mood detection, so this can be considered as met.
- Compatibility: The frontend user interface hosted on Roundcube is functional in Chrome, Safari and Firefox – I tested this during development.
- Security & Privacy: Data exchanged between the client and server is encrypted in transit via HTTPS, and temporary files, e.g., unsaved GIFs and images, are deleted after the user session ends. However, user data is not encrypted in the backend, making it vulnerable if the backend is compromised.
- Maintainability: The codebase is divided into modules that allow for future improvements. With clear documentation and a provided requirements list, it is easy for other developers to review my application.

10.2. Limitations and Future Improvements

- Currently, there is no backend database serving options such as animation archives or previous image analysis data. As a result, users can't reuse previously captured images or access older animations once the session ends without previously downloading them. The implementation of persistent storage to enable these features whilst accounting for privacy of data, can be done by encrypting the data in the backend.
- Flask is more suited to small-scale services; it becomes inefficient when required to handle concurrent processes, such as multiple users on the application, thereby making it harder to scale. In the future deploying the image processing engines on cloud databases will remove the reliance on local computational power, which could lead to inconsistent performance metrics across users. However, this can be very costly to implement and maintain, so this method serves its purpose in a development environment.
- To some extent, there is also a lack of automation in the frontend processes, with the user being required to trigger multiple workflows manually. Whilst this could be seen as a limitation, it was intentional to involve the user in the process. These manual inputs allow the user to better understand my program and the processes being carried out at each point. However, this may become tedious for veteran users; instead, the initial hands-on approach could be made into a tutorial phase that the user can deactivate once they are confident with their understanding and navigating my program.
- Whilst data is encrypted in transit using HTTPS, it is not at rest, leaving the data vulnerable to attack of the backend host server. To solve this, the future implementation of end-to-end encryption for sensitive data would be essential for production use.
- Considerably, the largest limitation of my program is the complexity of the configuration process and setup requirements. Running Roundcube, Flask and setting up preferred email clients for IMAP/SMTP requests requires technical vigour. This could be resolved by packaging my plugin and backend Flask server into an executable format.
- Finally, expanding the scope of my plugin beyond compatibility with RoundCube alone to other services such as Gmail or Outlook could improve the adoption of my system in professional workspaces I aimed to target. Roundcube's restriction on attachment file sizes means reduction animation quality must be reduced to comply, and the lack of support for in-line attachments in the composer means that the generated GIFs can

only be added as file attachments; overriding these will result in a higher quality application.

- The deep learning mood detection model used is only capable of picking up six basic emotions (anger, disgust, happy, sad, fear, surprise) and the neutral expression. However, new studies have found that humans may have as many as 27 emotions [42]. Future innovations in facial expression recognition models that can capture this new range of emotions may be adopted to increase accuracy.
- Carrying out extensive user testing in the future would be beneficial for gaining feedback.

11. Conclusion

To conclude, I created a facial expression animator with AI mood detection as my project title had dictated. This required extensive research into image processing techniques using deep learning models to facilitate mood and landmark detection, mathematical processes to gain an in-depth understanding of the animation component (e.g., Delaunay Triangulation for dividing faces into optimal triangular meshes).

I'd consider my project a success because I was able to meet the initial system requirements, both functional and non-functional, along with my aims and objectives. Additionally, I believe my application addressed key concerns participants in the research made about the solution I proposed, such as privacy and perception of unprofessionalism – the more difficult task, tackled by allowing the creation of custom in-line GIFs, that are specific to the context. However, there remain weaknesses in my system regarding scalability and multi-user support, as detailed above.

12. Bibliography and Citations

- [1] T. Hobbes, *Leviathan*. London, U.K.: Andrew Crooke, 1651.
- [2] "Employees Cite Lack of Collaboration for Workplace Failures," *Fierce*. <https://fierceinc.com/employees-cite-lack-of-collaboration-for-workplace-failures/>
- [3] "Communication Statistics 2025 | Project.co," *Project.co*, 2025. <https://www.project.co/communication-statistics/#section5> (accessed March. 11, 2025).
- [4] Microsoft, "What are IMAP and POP?," *support.microsoft.com*. <https://support.microsoft.com/en-us/office/what-are-imap-and-pop-ca2c5799-49f9-4079-aefe-ddca85d5b1c9>
- [5] "Darktrace," *Darktrace.com*, 2024. <https://www.darktrace.com/cyber-ai-glossary/simple-mail-transfer-protocol-smtp>
- [6] Wikipedia Contributors, "Flask (web framework)," *Wikipedia*, Apr. 27, 2019. [https://en.wikipedia.org/wiki/Flask_\(web_framework\)](https://en.wikipedia.org/wiki/Flask_(web_framework))
- [7] PLANEKS, "What Is Flask Used for? (9 Use Cases)," *PLANEKS*, Jan. 18, 2023. <https://www.planeks.net/what-is-flask-used-for/>
- [8] "About Roundcube Webmail," *roundcube.net*. <https://roundcube.net/about/>
- [9] "Py-Feat: Python Facial Expression Analysis Toolbox — Py-Feat," *py-feat.org*. <https://py-feat.org/pages/intro.html>
- [10] "Article," *Linkedin.com*, 2025. <https://www.linkedin.com/advice/1/what-best-ways-use-image-morphing-engaging-visual-jc78e>
- [11] A. Wang, "Face Morphing — A Step-by-step Tutorial with Code," *Medium*, Aug. 08, 2020. <https://azmariewang.medium.com/face-morphing-a-step-by-step-tutorial-with-code-75a663cdc666>
- [12] W. Wang, Y. Zhao, L. Qiu, and Y. Zhu, "Effects of Emoticons on the Acceptance of Negative Feedback in Computer-Mediated Communication," *AIS Electronic Library (AISel)*, 2025. <https://aisel.aisnet.org/jais/vol15/iss8/3/>
- [13] M. A. Riordan and E. Glikson, "On the Hazards of the Technology Age: How Using Emojis Affects Perceptions of Leaders," *International Journal of*

Business Communication, p. 232948842097169, Nov. 2020, doi:
<https://doi.org/10.1177/2329488420971690>.

[14] L. Boissoneault, "A Brief History of the GIF, From Early Internet Innovation to Ubiquitous Relic," *Smithsonian*, Jun. 02, 2017.
<https://www.smithsonianmag.com/history/brief-history-gif-early-internet-innovation-ubiquitous-relic-180963543/>

[15] S. Karre, "GIFs and their Effects on Communication Skills," *SSRN Electronic Journal*, 2025, doi: <https://doi.org/10.2139/ssrn.5021118>.

[16] K. M. Miltner and T. Highfield, "Never Gonna GIF You Up: Analyzing the Cultural Significance of the Animated GIF," *Social Media + Society*, vol. 3, no. 3, p. 205630511772522, Jul. 2017, doi:
<https://doi.org/10.1177/2056305117725223>.

[17] E. Shandilya, M. Fan, and G. W. Tigwell, "'I need to be professional until my new team uses emoji, GIFs, or memes first': New Collaborators' Perspectives on Using Non-Textual Communication in Virtual Workspaces," *CHI Conference on Human Factors in Computing Systems*, Apr. 2022, doi: <https://doi.org/10.1145/3491102.3517514>.

[18] Flask, "Welcome to Flask — Flask Documentation (3.0.x)," *Palletsprojects.com*, 2010. <https://flask.palletsprojects.com/en/stable/>

[19] G. W. Lucas, "Delaunay Triangulation," *gwlucastrig.github.io*.
<https://gwlucastrig.github.io/TinfourDocs/DelaunayIntro/index.html>

[20] OpenCV, "About OpenCV," *OpenCV*, 2018. <https://opencv.org/about/>

[21] imageio contributors, "imageio: Library for reading and writing a wide range of image, video, scientific, and volumetric data formats.," *PyPI*.
<https://pypi.org/project/imageio/>

[22] O. R. Musin, "Properties of the Delaunay triangulation," *Symposium on Computational Geometry*, Aug. 1997, doi:
<https://doi.org/10.1145/262839.263061>.

[23] "Base64 - MDN Web Docs Glossary: Definitions of Web-related terms | MDN," *developer.mozilla.org*. <https://developer.mozilla.org/en-US/docs/Glossary/Base64>

[24] I. Özkan, "Data Protection Principles: The 7 Principles Of GDPR Explained," *www.cyberpilot.io*, Nov. 2022.
<https://www.cyberpilot.io/cyberpilot-blog/data-protection-principles-the-7-principles-of-gdpr-explained/>

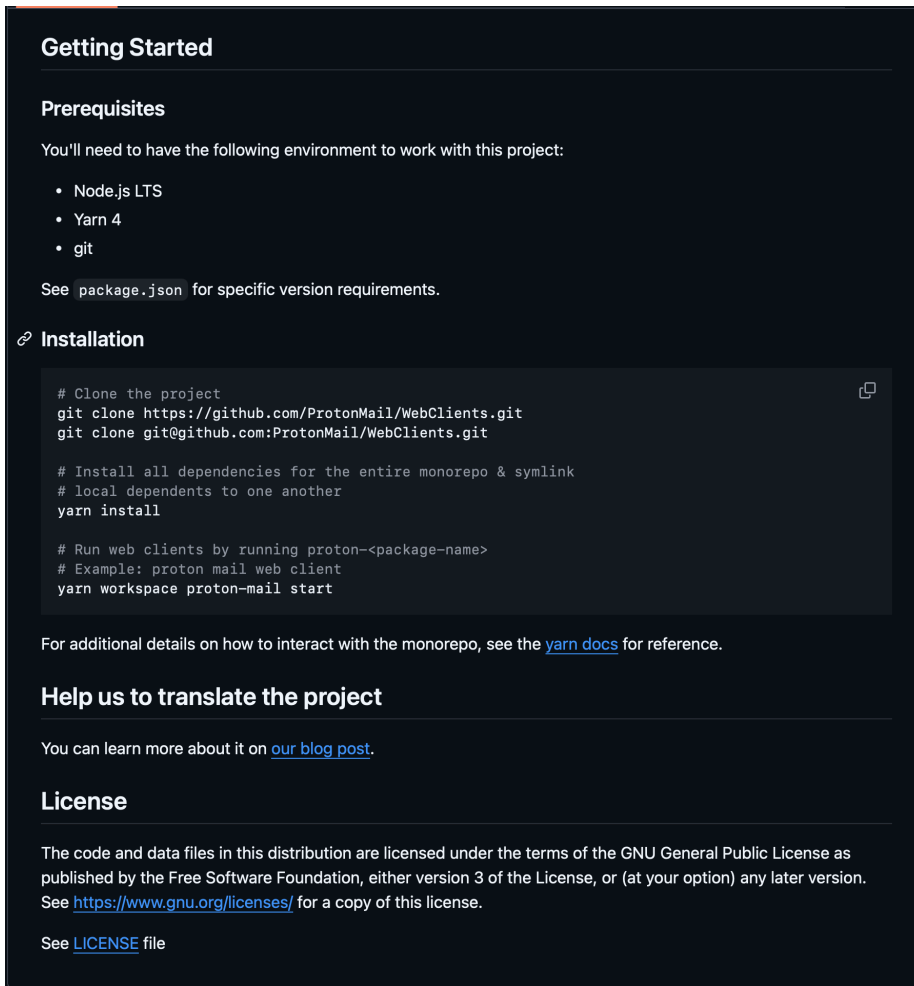
- [25] M. Conti, N. Dragoni, and V. Lesyk, "A Survey of Man In The Middle Attacks," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 3, pp. 2027–2051, 2016, doi: <https://doi.org/10.1109/comst.2016.2548426>.
- [26] S. Laoyan, "What is agile methodology? (A beginner's guide)," *Asana*, Feb. 20, 2025. <https://asana.com/resources/agile-methodology>
- [27] Cloudflare, "What is HTTPS? | Cloudflare UK," *Cloudflare*, 2024. Available: <https://www.cloudflare.com/en-gb/learning/ssl/what-is-https/>
- [28] J. Dunbar, "The Impact of Poor Communication in the Workplace and How to Fix It," *Concretecms.com*, 2024. <https://www.concretecms.com/about/blog/intranets/the-impact-of-poor-communication>
- [29] Roundcube, "GitHub - roundcube/roundcubemail: The Roundcube Webmail suite," *GitHub*, Feb. 08, 2025. <https://github.com/roundcube/roundcubemail>
- [30] Jin Hyun Cheong, T. Xie, S. Byrne, and L. J. Chang, "Py-Feat: Python Facial Expression Analysis Toolbox," *Affective Science*, Aug. 2023, doi: <https://doi.org/10.1007/s42761-023-00191-4>.
- [31] L. Pham, T. H. Vu, and T. A. Tran, "Facial Expression Recognition Using Residual Masking Network," *IEEE Xplore*, Jan. 01, 2021. <https://ieeexplore.ieee.org/document/9411919/>
- [32] phamquiluan, "GitHub - phamquiluan/ResidualMaskingNetwork: ICPR 2020: Facial Expression Recognition using Residual Masking Network," *GitHub*, Mar. 23, 2022. <https://github.com/phamquiluan/ResidualMaskingNetwork>
- [33] S. Chen, Y. Liu, X. Gao, and Z. Han, "MobileFaceNets: Efficient CNNs for Accurate Real- Time Face Verification on Mobile Devices." Available: <https://arxiv.org/pdf/1804.07573>
- [34] "Image Warping and Morphing," *GeeksforGeeks*, Oct. 05, 2023. <https://www.geeksforgeeks.org/image-warping-and-morphing/>
- [35] OpenAI, "ChatGPT," *chatgpt.com*, 2025. <https://chatgpt.com>
- [36] "2. Geometric Transformations — Computer Vision," *Github.io*, 2023. https://hatefdastour.github.io/notes/Computer_Vision/CV_C2.html

- [37] "Using Fetch," MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- [38] "Tutorial — Cryptography 3.4.6 documentation," *Cryptography.io*, 2021. <https://cryptography.io/en/3.4.6/x509/tutorial.html>
- [39] Cloudflare, "What is an SSL certificate?," *Cloudflare.com*, 2014. <https://www.cloudflare.com/en-gb/learning/ssl/what-is-an-ssl-certificate/>
- [40] N. Gul, "Use SSL Certificates from Public CA or Create a Private CA?," *Medium*, May 2023. <https://paktek123.medium.com/use-ssl-certificates-from-public-ca-or-create-a-private-ca-f83d13714394>
- [41] ProtonMail, "GitHub - ProtonMail/WebClients: Monorepo hosting the proton web clients," *GitHub*, Apr. 12, 2024. <https://github.com/ProtonMail/WebClients>
- [42] "What Are the 27 Basic Emotions?," *MedicineNet*. https://www.medicinenet.com/what_are_the_27_basic_emotions/article.html

Additional readings:

- [43] Visualizing Facial Expressions — Py-Feat," *Py-feat.org*, 2022. https://py-feat.org/basic_tutorials/03_plotting.html
- [44] "(PDF) Online versus face-to-face conversation: An examination of relational and discourse variables.," *ResearchGate*. https://www.researchgate.net/publication/232604715_Online_versus_face-to-face_conversation_An_examination_of_relational_and_discourse_variables
- [45] "How to Detect Emotions in Images Using Python," *Edlitera*. <https://www.edlitera.com/blog/posts/emotion-detection-in-images>
- [46] "Local development: cannot get past initial login screen · Issue #343 · ProtonMail/WebClients," *GitHub*, 2025. <https://github.com/ProtonMail/WebClients/issues/343>
- [47] *Github.com*, 2025. <https://github.com/ProtonMail/WebClients/issues/424>
- [47] "Reddit - The heart of the internet," *Reddit.com*, 2024. https://www.reddit.com/r/ProtonPass/comments/1dd1bzm/cant_run_proton_desktop_or_web_apps_from_source/

13. Appendix



Getting Started

Prerequisites

You'll need to have the following environment to work with this project:

- Node.js LTS
- Yarn 4
- git

See `package.json` for specific version requirements.

Installation

```
# Clone the project
git clone https://github.com/ProtonMail/WebClients.git
git clone git@github.com:ProtonMail/WebClients.git

# Install all dependencies for the entire monorepo & symlink
# local dependents to one another
yarn install

# Run web clients by running proton-<package-name>
# Example: proton mail web client
yarn workspace proton-mail start
```

For additional details on how to interact with the monorepo, see the [yarn docs](#) for reference.

Help us to translate the project

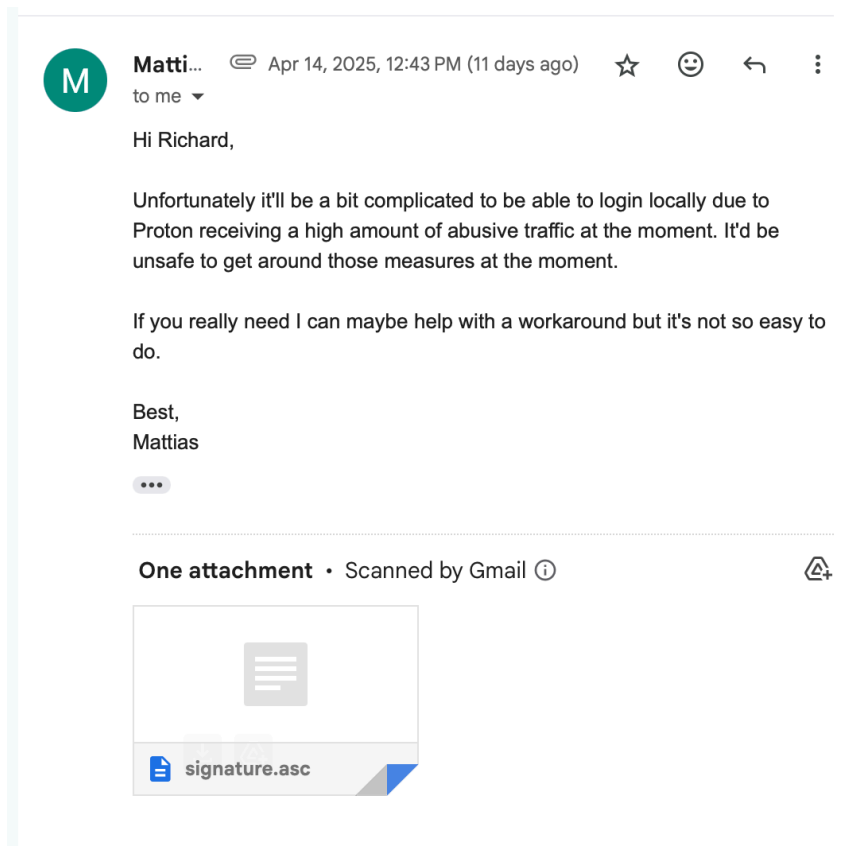
You can learn more about it on [our blog post](#).

License

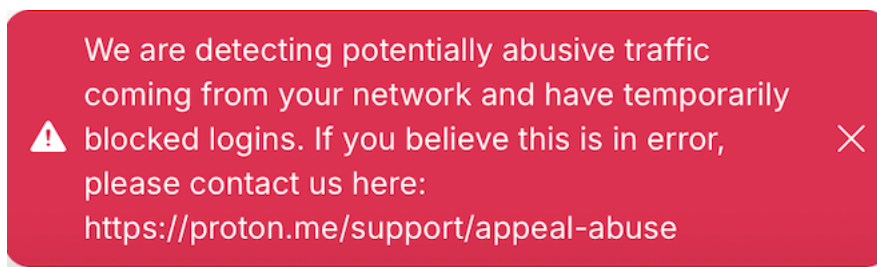
The code and data files in this distribution are licensed under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version. See <https://www.gnu.org/licenses/> for a copy of this license.

See [LICENSE](#) file

Appendix A: ProtonMail README File Screenshot



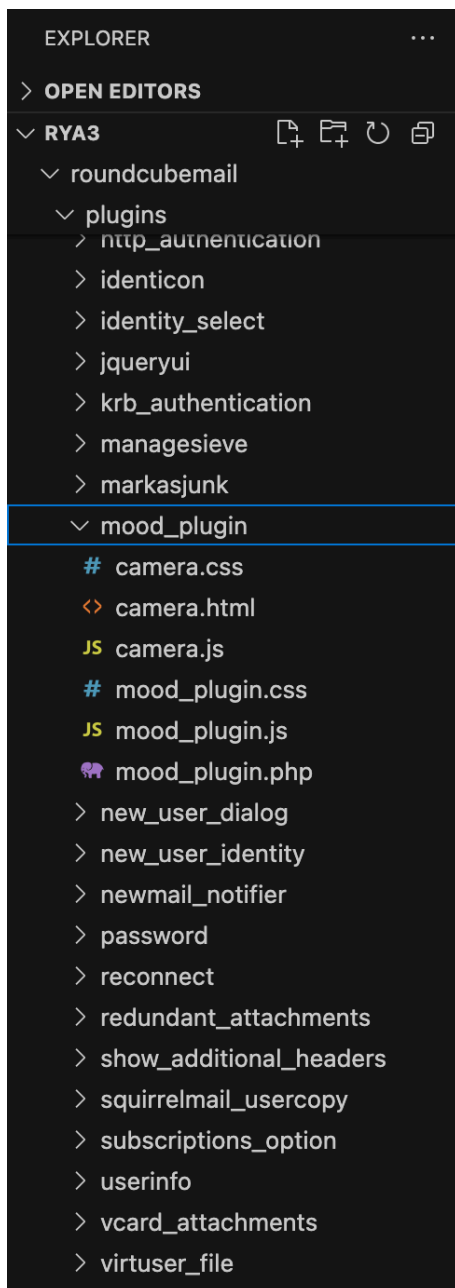
Appendix B: Conversation with ProtonMail Dev.



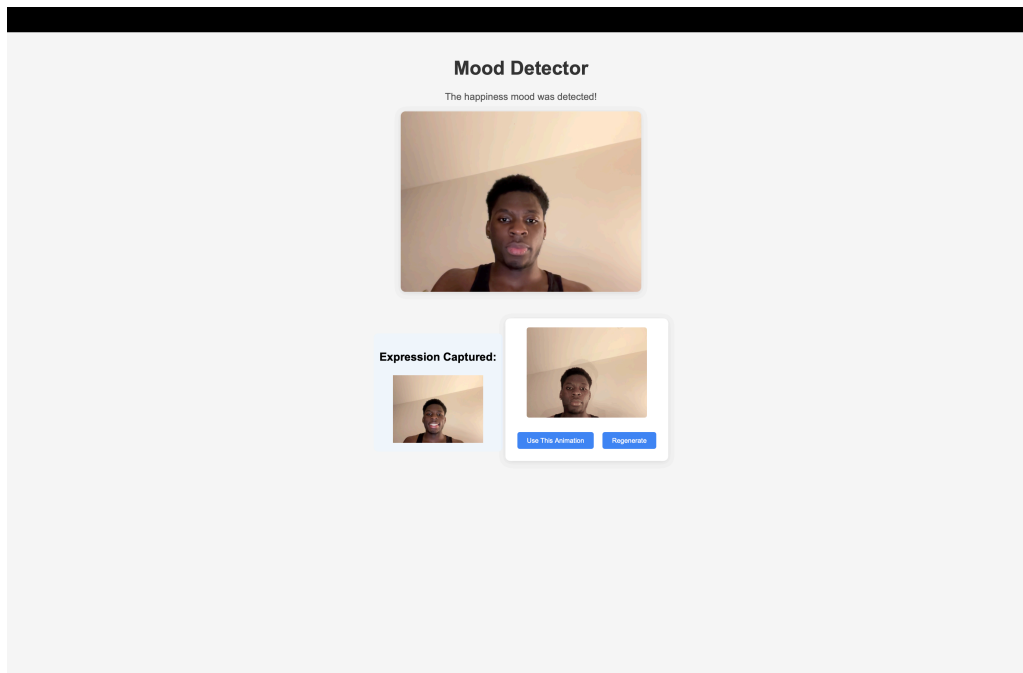
Appendix C: Screenshot of Abusive Traffic notification



Appendix D: SSL certificate verification error



Appendix E: Roundcube plugin folder structure with custom mood plugin



Appendix F. Mood Capture Web UI

Appendix G. Creating and using a self-signed certificate

```
# Generate self-signed certificate
def generate_self_signed_cert(cert_path, key_path):
    # Generate a private key
    key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048,
    )
    # write the private key to a file
    with open(key_path, "wb") as key_file:
        key_file.write(key.private_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PrivateFormat.TraditionalOpenSSL,
            encryption_algorithm=serialization.NoEncryption()
        ))
    # issuer details
    subject = issuer = x509.Name([
        x509.NameAttribute(NameOID.COUNTRY_NAME, u"UK"),
        x509.NameAttribute(NameOID.STATE_OR_PROVINCE_NAME, u"London"),
        x509.NameAttribute(NameOID.LOCALITY_NAME, u"London"),
        x509.NameAttribute(NameOID.ORGANIZATION_NAME, u"Mood Plugin"),
        x509.NameAttribute(NameOID.COMMON_NAME, u"localhost"),
    ])
    # Create a self-signed certificate
    cert = (
        x509.CertificateBuilder()
        .subject_name(subject)
```

```

        .issuer_name(issuer)
        .public_key(key.public_key())
        .serial_number(x509.random_serial_number())
        .not_valid_before(datetime.utcnow())
        .not_valid_after(datetime.utcnow() + timedelta(days=365))
        .add_extension(
            x509.SubjectAlternativeName([x509.DNSName(u"localhost")]),
            critical=False,
        )
        .sign(key, hashes.SHA256())
    )
    # Write the certificate to a file
    with open(cert_path, "wb") as cert_file:
        cert_file.write(cert.public_bytes(serialization.Encoding.PEM))

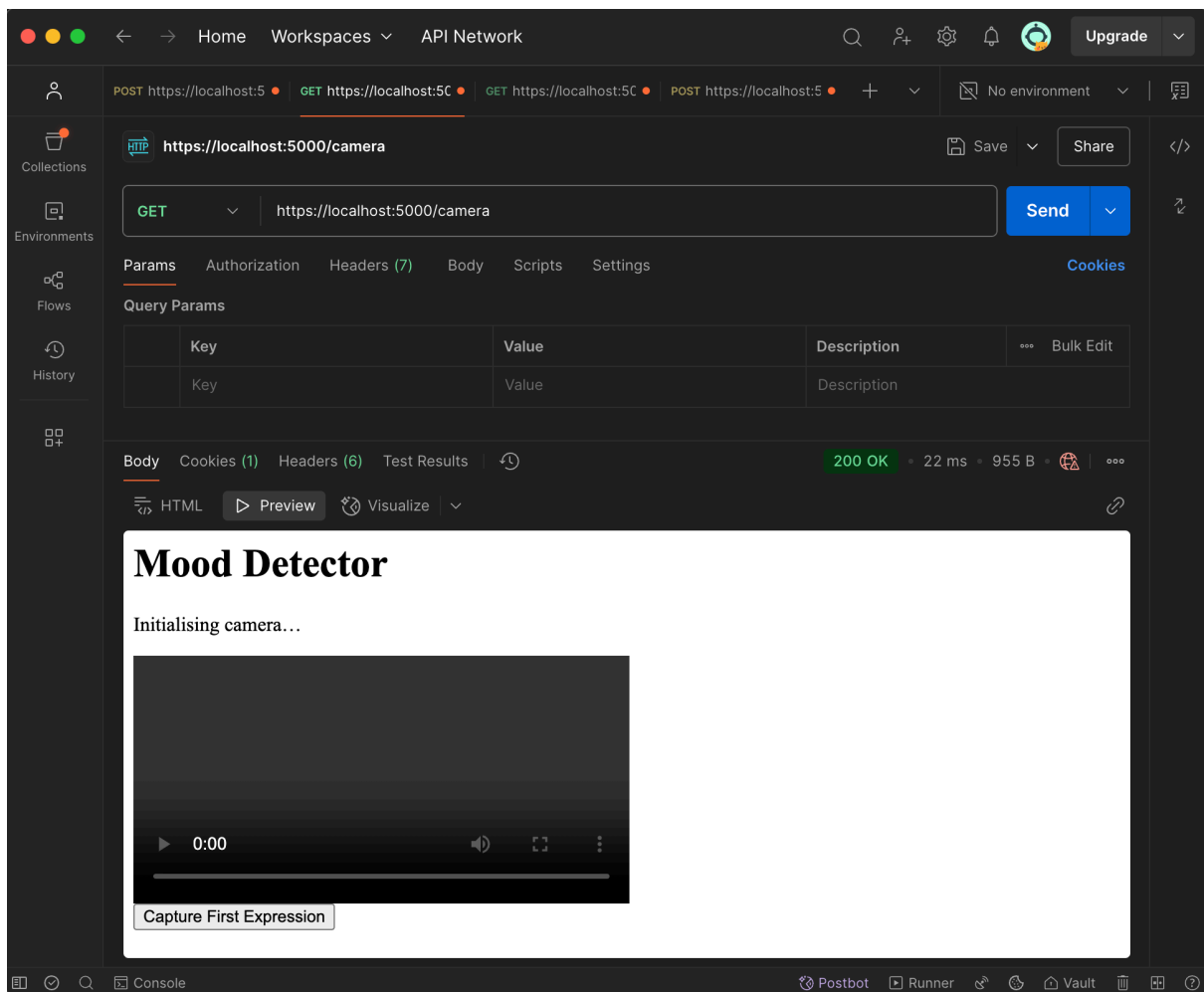
#Ensure no existing certificate and key
if not os.path.exists(CERT_FILE) or not os.path.exists(KEY_FILE):
    generate_self_signed_cert(CERT_FILE, KEY_FILE)
    logger.info(f"Generated self-signed certificate at {CERT_FILE} and key at {KEY_FILE}")
else:
    logger.info(f"Certificate and key already exist at {CERT_FILE} and {KEY_FILE} reusing...")

ssl_context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
ssl_context.load_cert_chain(certfile=CERT_FILE, keyfile=KEY_FILE)

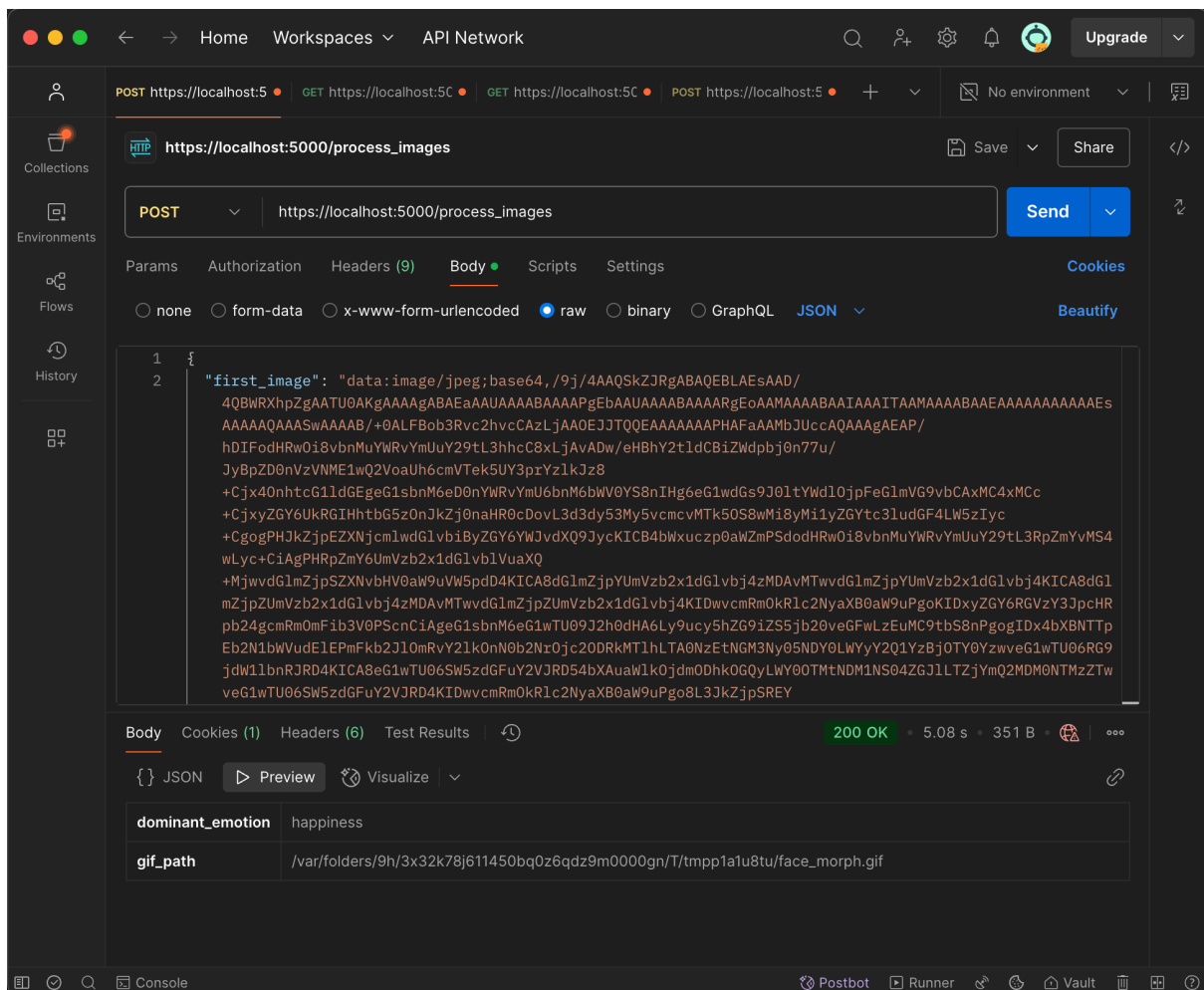
.....
if __name__ == '__main__':
    try:
        app.run(host='0.0.0.0', port=5000, debug=True, ssl_context=ssl_context)
    finally:
        shutil.rmtree(temp_dir, ignore_errors=True)

```

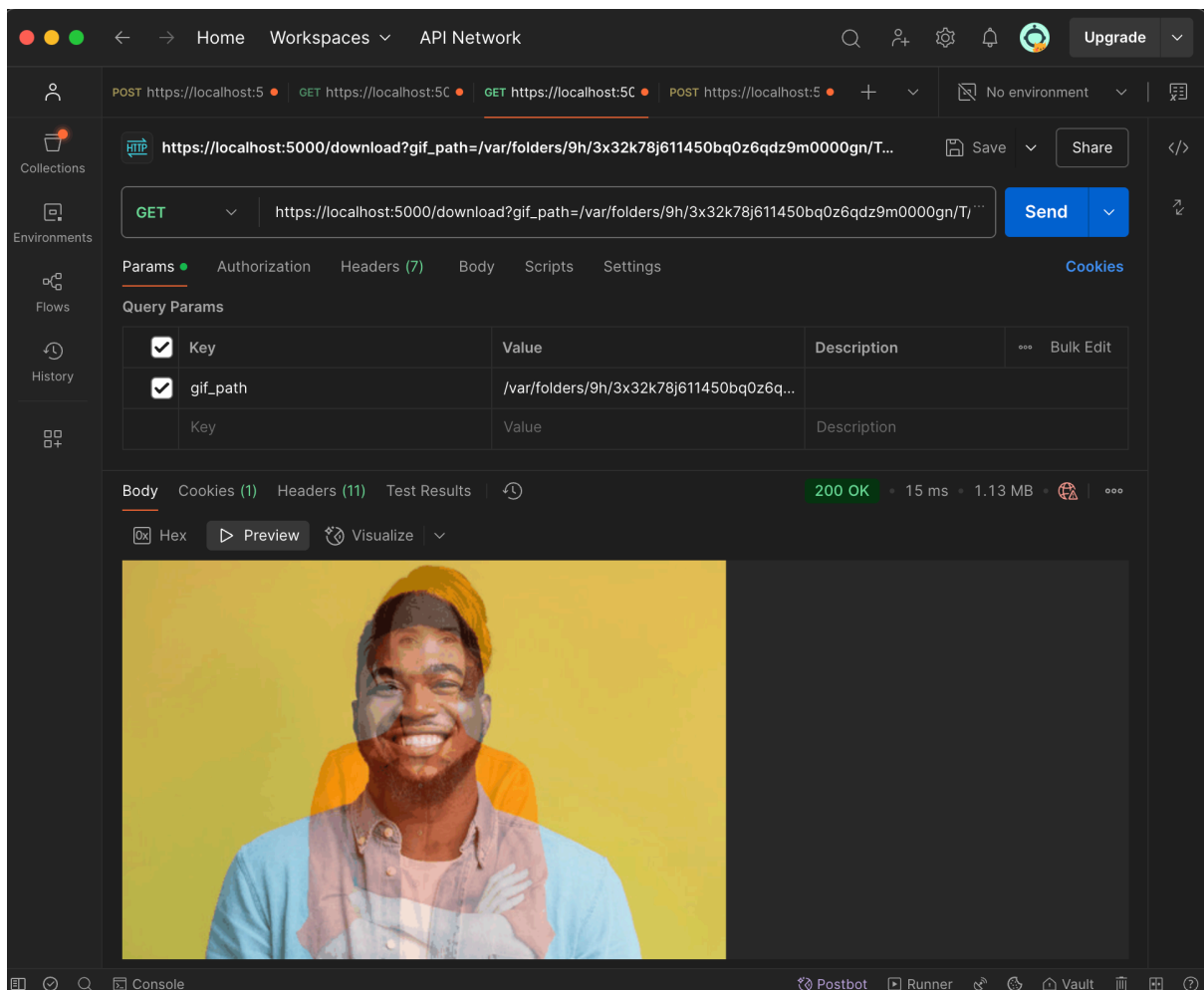
Appendix G. Creating and using a self-signed certificate



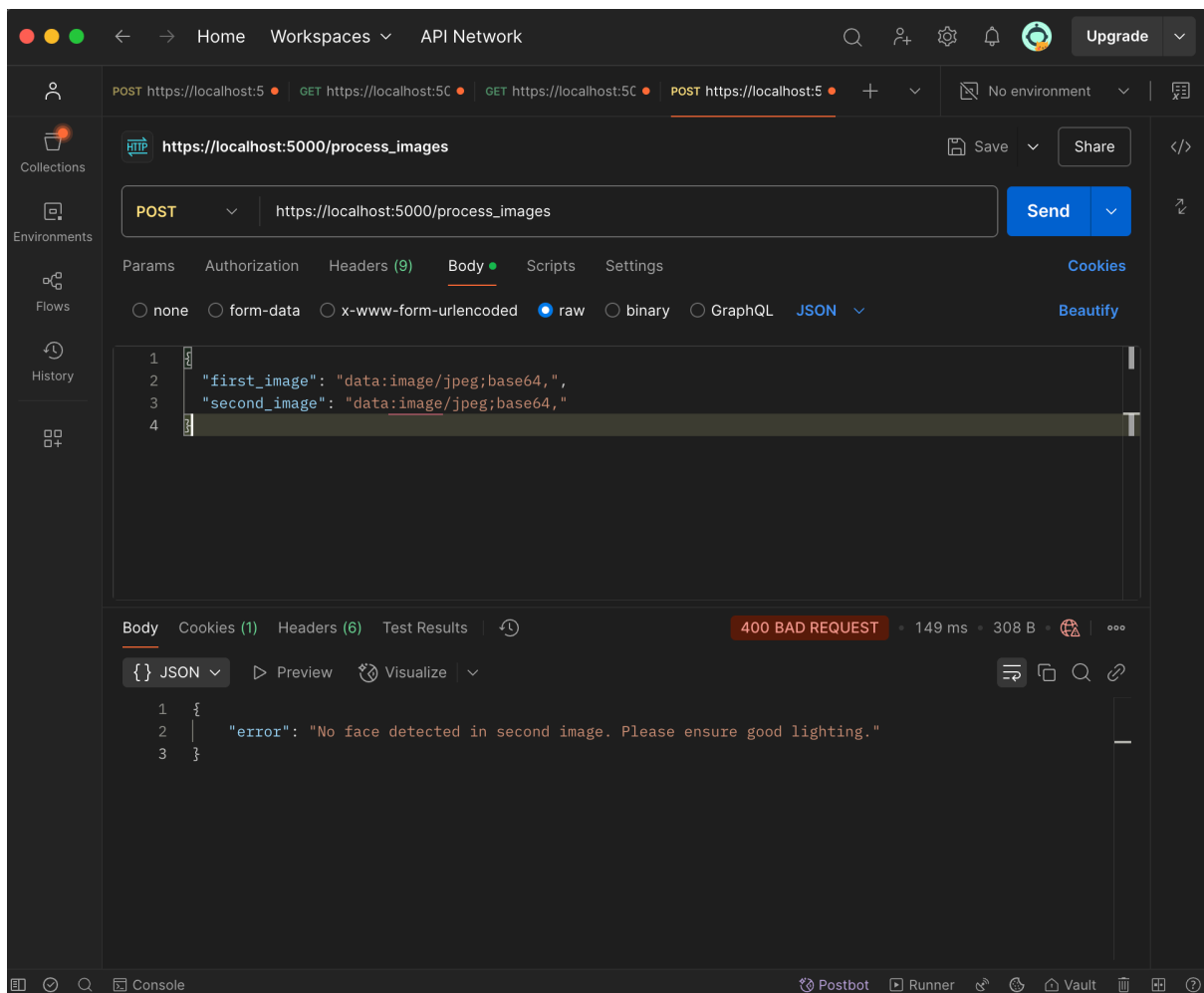
Appendix H. Postman /capture response



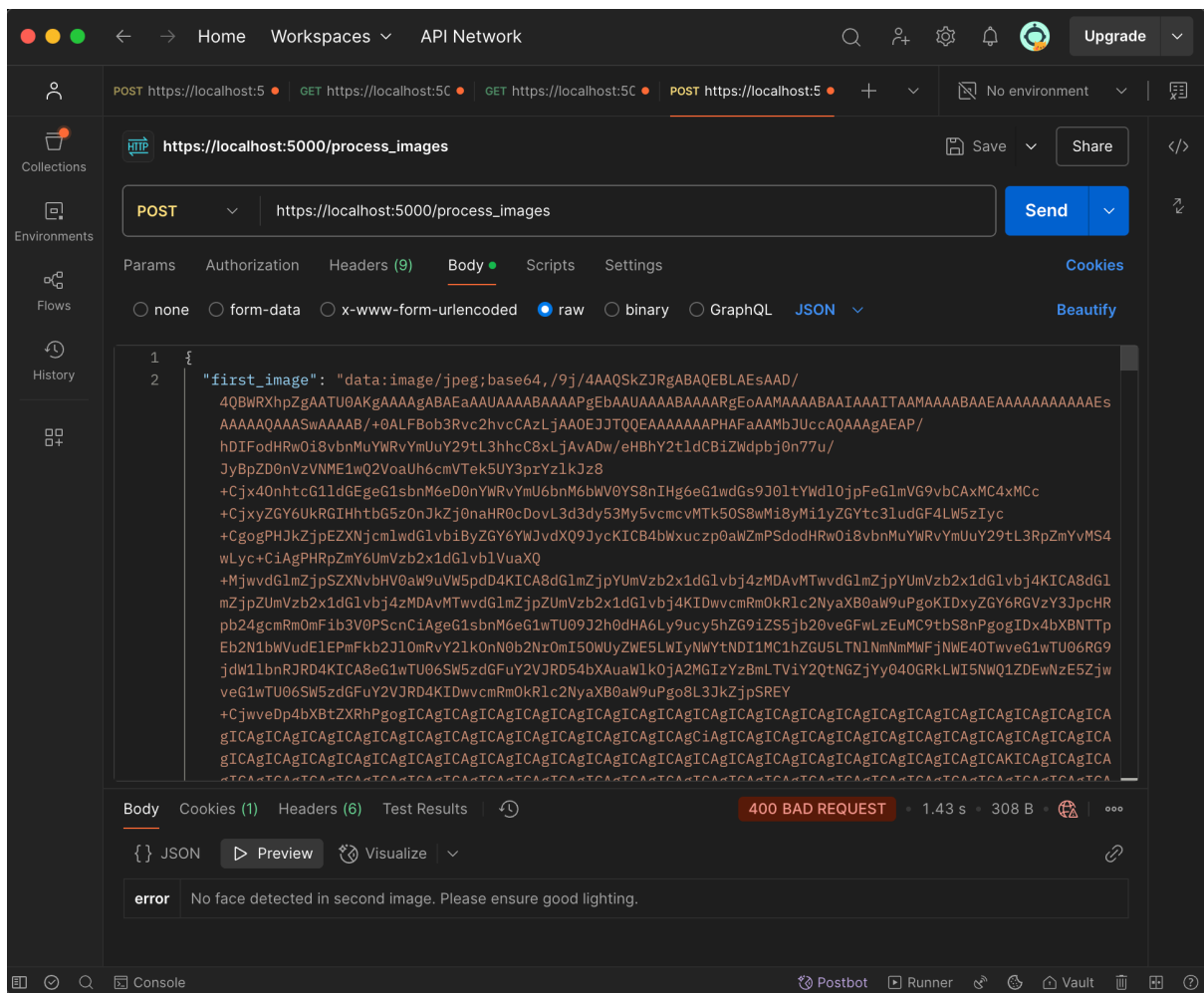
Appendix I. Postman /process_images response



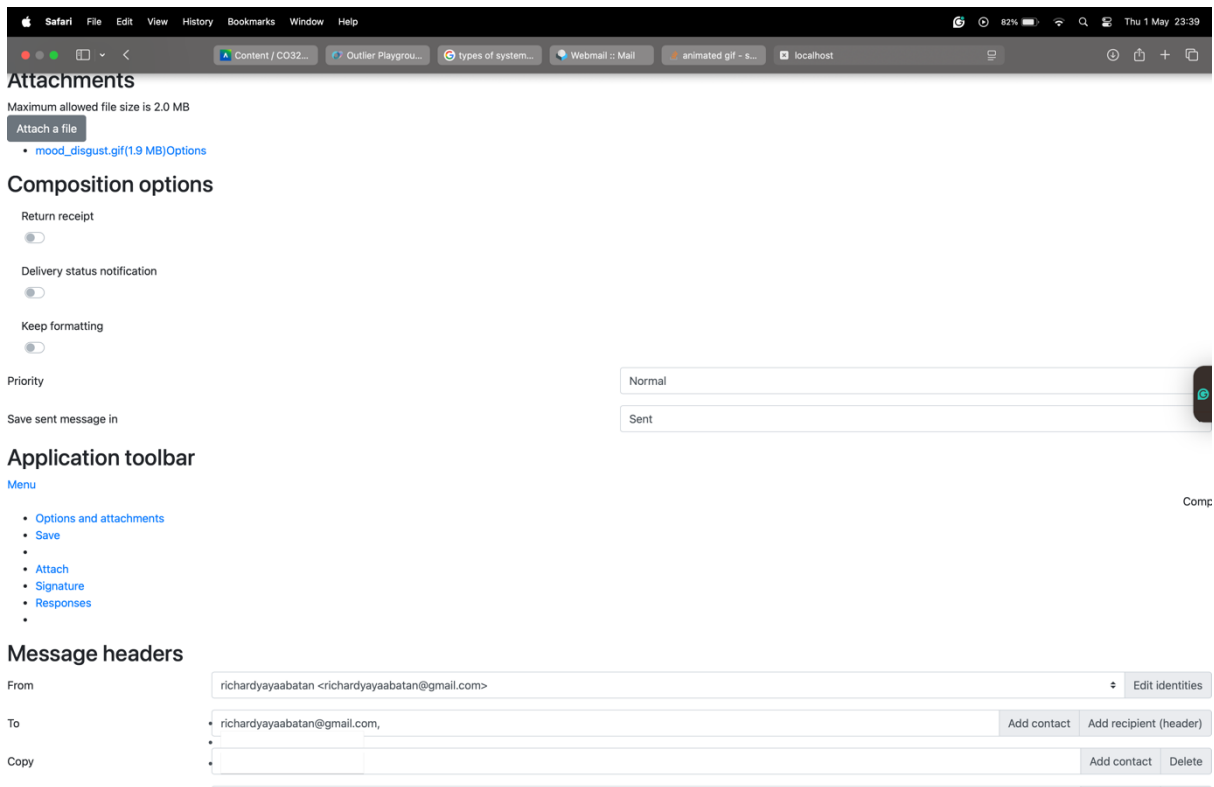
Appendix J. Postman /download endpoint response



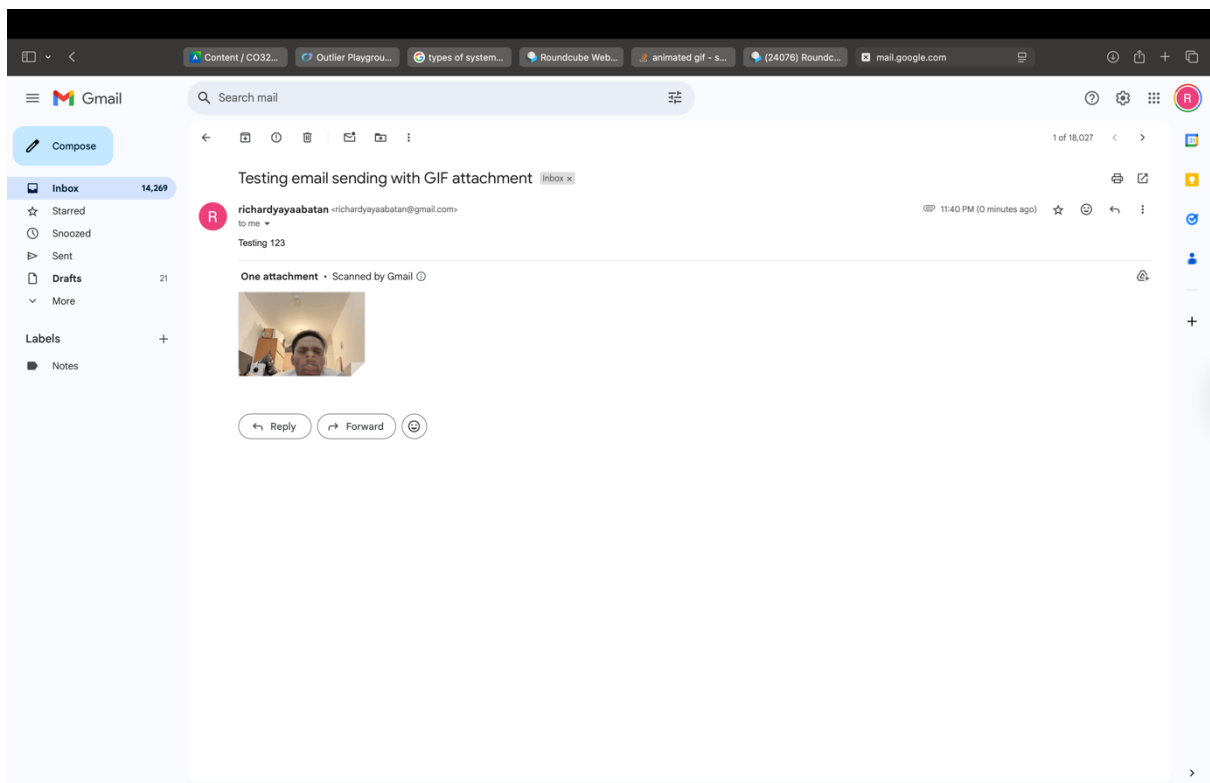
Appendix K. Postman /process_image empty image response



Appendix L. Postman /process_image household images response



Appendix M. Outgoing email with GIF attachment



Appendix N. Incoming email with GIF attachment